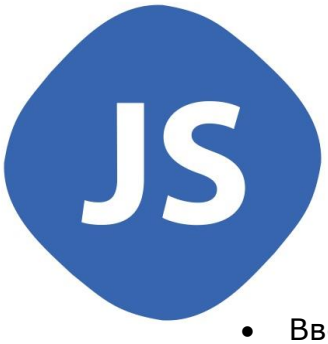


HTTP протокол

- Введение
- HTTP-запросы
- Структура HTTP-запроса
- Структура HTTP-ответа
- Заголовки в запросах HTTP

Введение

HTTP протокол

HTTP означает **протокол передачи гипертекста** (**HyperText Transfer Protocol**). Он представляет собой список правил, по которым компьютеры обмениваются данными в Глобальной сети Интернет. HTTP умеет передавать все возможные форматы файлов — текст, аудио, видео. Но при этом состоит только из текста.

Например, когда пользователь вводит в адресной строке браузера **https://vk.com/pechora_pro**, он:

- **составляет** запрос;
- **отправляет** его на сервер, чтобы получить HTML-страницу сообщества.

Когда сервер обрабатывает запрос, то он отправляет ответ, в котором сообщает, что такой ресурс найден (!;) и загружает страницу в браузер.

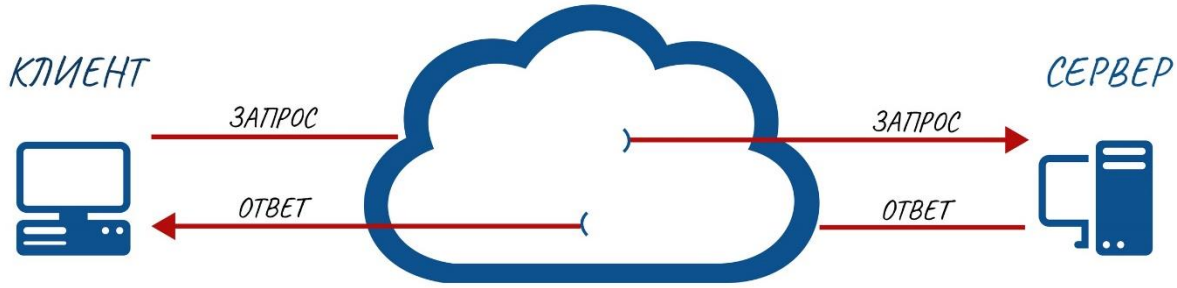
Кроме HTTP в интернете работает протокол **HTTPS**. Аббревиатура расшифровывается как **защищённый протокол передачи гипертекста** (**HyperText Transfer Protocol Secure**). Он нужен для безопасной передачи данных по сети. Обмен запросами происходит по тем же принципам, как и у HTTP, но, перед отправкой данные дополнительно шифруются, а затем расшифровываются на сервере.



HTTPS используют во время ввода данных банковской карты или паролей на сайтах — в целом большинство современных сайтов используют именно его.

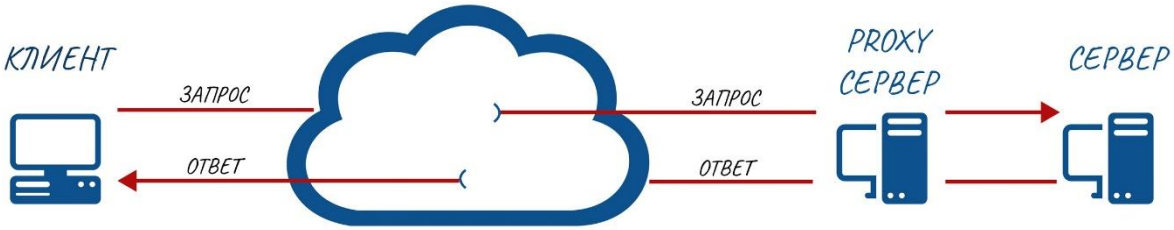
Из чего состоит HTTP

HTTP состоит из двух основных элементов: **клиента** и **сервера**. Клиент отправляет запросы и ждёт данные от сервера. Сервер ждёт, пока ему придёт очередной запрос, обрабатывает его и возвращает ответ клиенту.

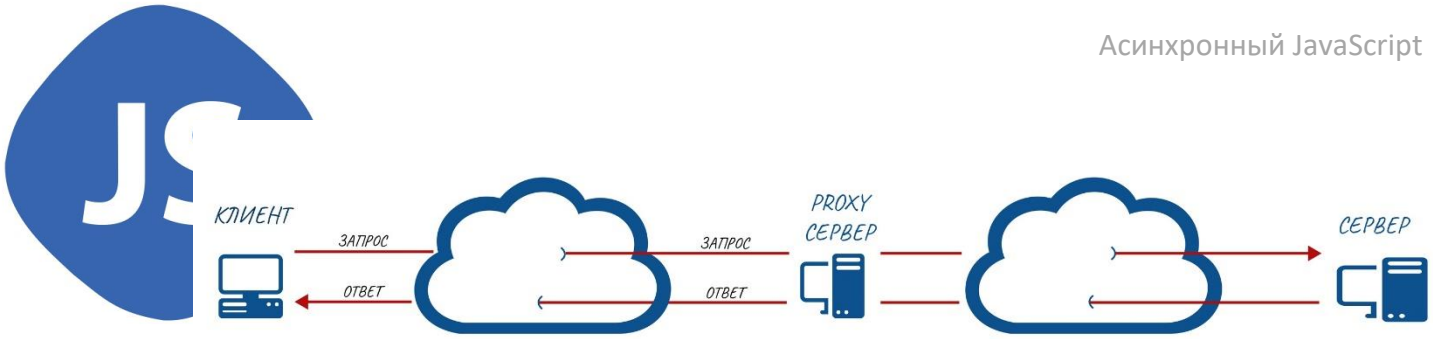


Обычно связь между клиентом и сервером имеет посредника в виде **прокси-сервера**. Прокси сервер нужен для разных операций — например, для безопасности и конфиденциальности, кэширования или распределения нагрузки на обслуживаемые серверы.

Типичная процедура отправки запроса от клиента серверу выглядит так:



Или так:



Клиентом может быть любое устройство, через которое пользователь запрашивает данные. Часто в роли клиента выступает **веб-браузер** (**браузер**), программы для отладки приложений или даже командная строка. Главная особенность клиента — он всегда **инициирует запрос**.

Сервер — это устройство, которое **обрабатывает запросы клиента**. Он может состоять как из одного компьютера, так и из кластера. Несколько виртуальных серверов могут находиться на одной физической машине.

Прокси-сервер — это один или несколько **второстепенных серверов**, которые располагаются между клиентом и главным сервером. Они обрабатывают запросы, а также ответы на них. Чаще всего прокси-серверы используют для кэширования и сжатия данных, обхода ограничений и анонимных запросов.

Структура HTTP-запроса

В общем виде HTTP-запрос состоит из нескольких частей:

- **Стартовая строка**. Задаёт параметры запроса или ответа.
- **Произвольный набор HTTP заголовков**. Каждый заголовок определяет служебную информацию запроса (ответа) или описывает тело сообщения.
- **Пустая строка**. Указывает, что вся служебная информация отправлена.
- **Тело запроса**. Обычно в нём находятся передаваемые данные. От заголовков тело отделяется пустой строкой.

Стартовая строка называется:

- в запросе – **строкой запроса** (**Request Line**);

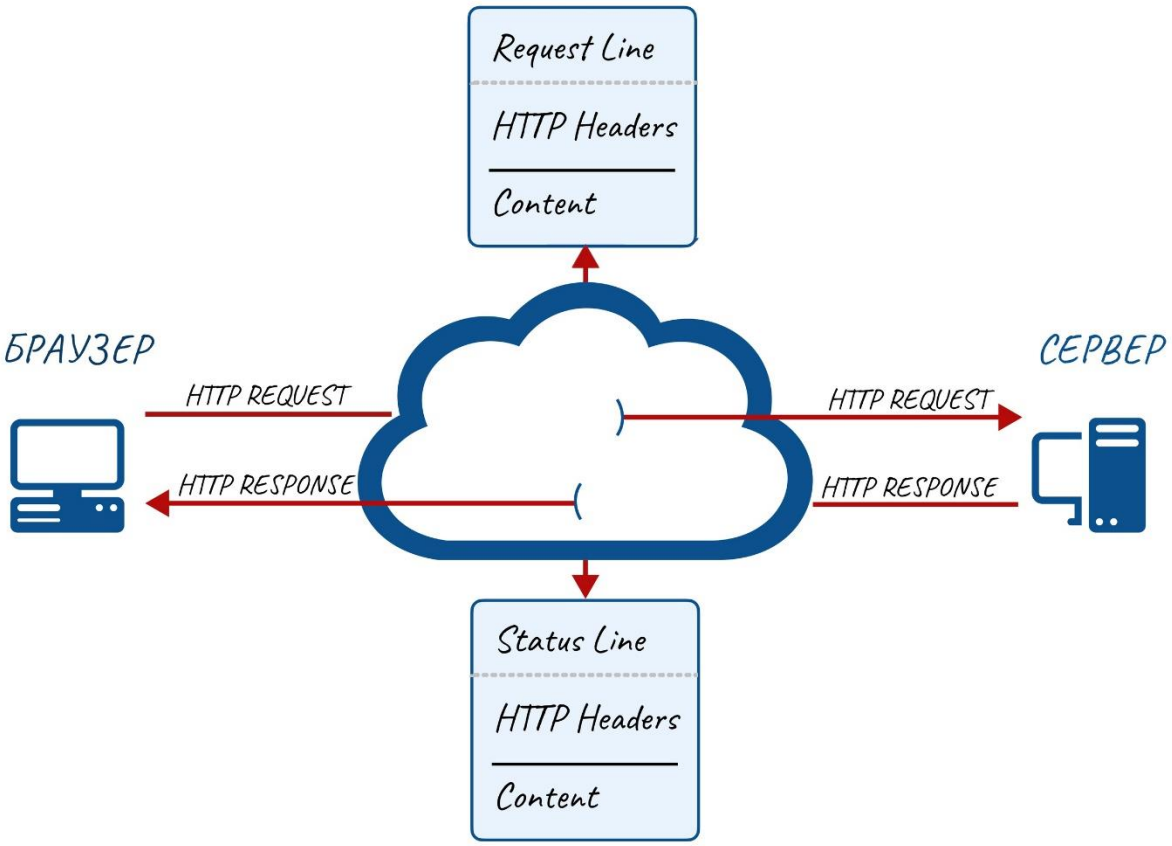
- в ответе – **строкой состояния** (**Status Line**).

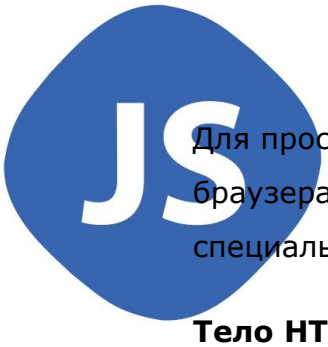
Стартовая строка является особым видом заголовка.

Заголовки HTTP (**HTTP Headers**) позволяют клиенту и серверу обмениваться **дополнительной информацией** с HTTP запросом или ответом. В HTTP-заголовке содержится не чувствительное к регистру название, а затем после (:) непосредственно значение. Пробелы перед значением игнорируются.

HTTP-заголовки сопровождают обмен данными по **протоколу HTTP**. Они могут содержать описание данных и несут информацию о браузере клиента, запрошенной странице, сервере и многом другом и способствуют **взаимодействию между клиентом и сервером**.

При открытии любой веб-страницы браузер отправляет десятки **HTTP-заголовков** и получает **HTTP-ответы** для каждого из них.





Для просмотра HTTP-заголовков достаточно включить **режим разработчика** браузера (F12). Для более глубокого анализа заголовков можно использовать специальные браузерные расширения.

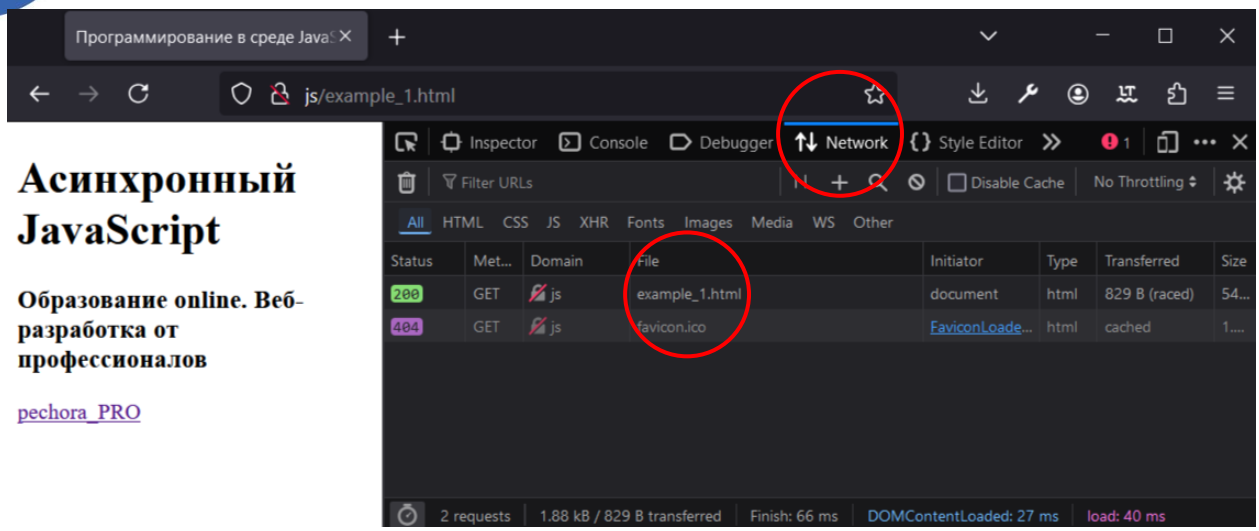
Тело HTTP-запроса – это байты данных, передаваемые в сообщении после пустой строки, отделяющей тело от заголовков. Большинство HTTP-запросов представляют собой GET-запросы без тела.

Первые эксперименты с запросами выполним в браузере **FireFox**.

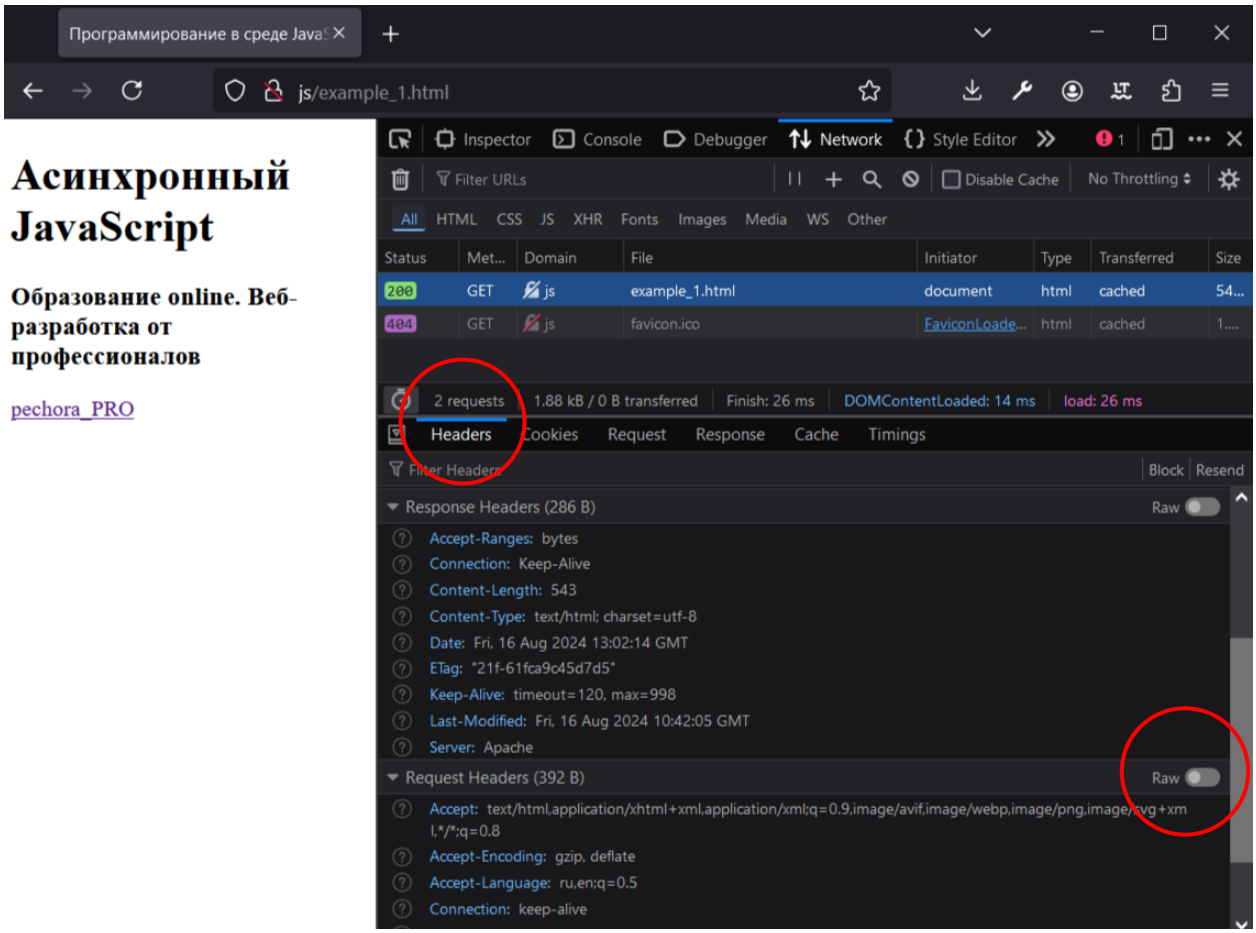
example_1.

```
<!DOCTYPE html>
<html lang="ru">
    <head>
        <meta http-equiv="Content-Type" content="text/html;
        charset=utf-8">
        <title>Программирование в среде JavaScript</title>
    </head>
    <body>
        <header align="center">
            <h1>Асинхронный JavaScript</h1>
        </header>
        <footer align="center">
            <h3>Образование online. Веб-разработка от
            профессионалов</h3>
            <a href="https://vk.com/pechora_pro"
            target="_blank">pechora_PRO</a>
        </footer>
    </body>
</html>
```

В режиме разработчика, перейдите на вкладку **Network** и **кликните** по названию запрашиваемого файла. Если вы сначала запросили файл, а уже потом перешли в режим разработчика – **перезагрузите** страницу.



Перейдите на вкладку просмотра заголовков (вкладка **Headers**).

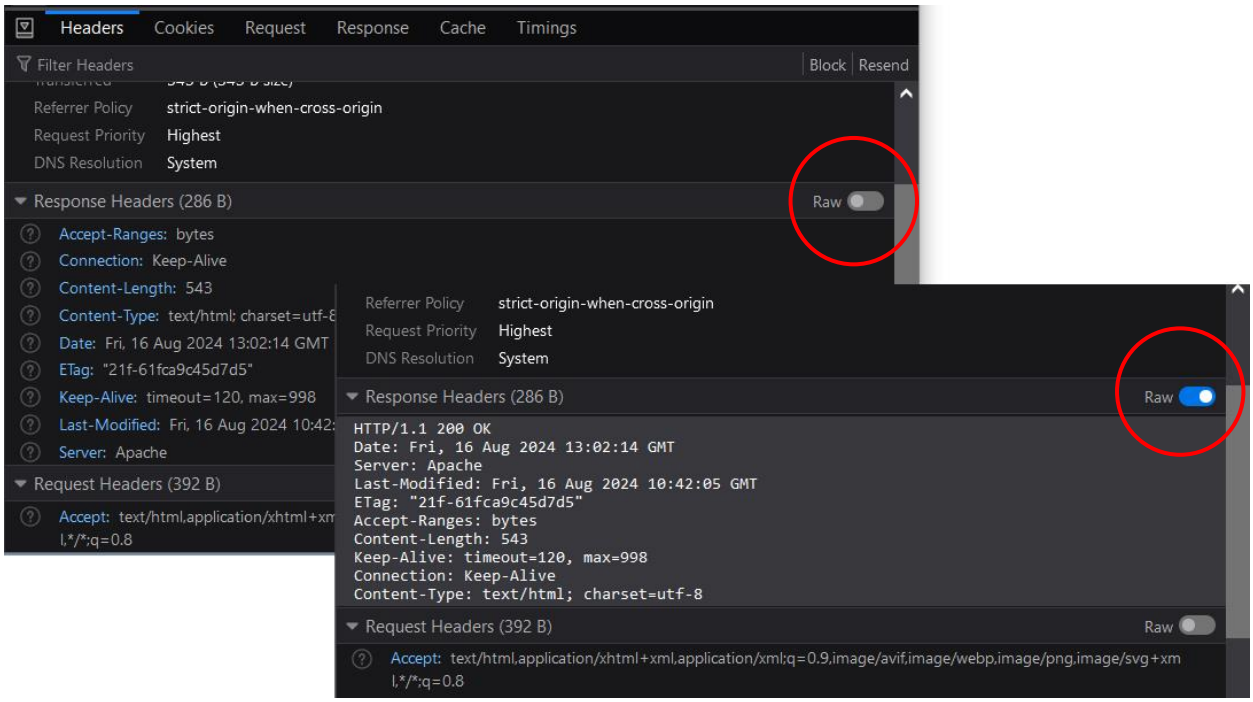


Обратите внимание на переключатель режимов **raw**.

Примечание. **RAW** (англ. raw — сырой, необработанный) — формат данных, содержащий необработанные (или обработанные в минимальной степени) данные, что позволяет избежать потерь информации, не имеющей чёткой спецификации.

Режим позволяет просмотреть заголовки в **сыром** виде – так, как их читает **браузер** и в **форматном** виде – так, как их удобно анализировать **разработчику**.

Разница в отображении заголовков наглядно продемонстрирована на следующем скриншоте.



Примечание. Приведенные в уроке коды демонстрационных примеров не совсем корректны с точки зрения изучения HTTP-запросов. В некоторых примерах хорошо бы не переходить по ссылке, а запрашивать ресурсы в фоновом режиме.

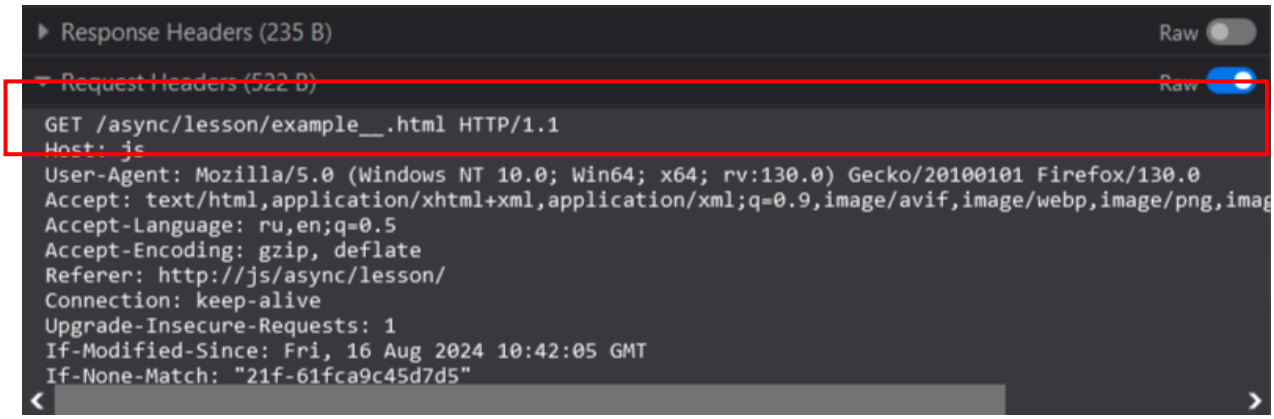
Асинхронность в запросах расставит все на свои места.

HTTP-запросы

Раздел содержит **общую** информацию о запросах и ответах сервера. Более подробный разбор программного кода будет приведён дальше по тексту урока.

Отправка HTTP-запроса

Когда пользователь вводит **URL-адрес внешнего ресурса** в адресной строке, браузер отправляет **HTTP-запрос**. Запрос может выглядеть следующим образом:



Первая строка запроса - это **Request Line** (**Строка запроса**), которая содержит некоторую базовую информацию запроса:

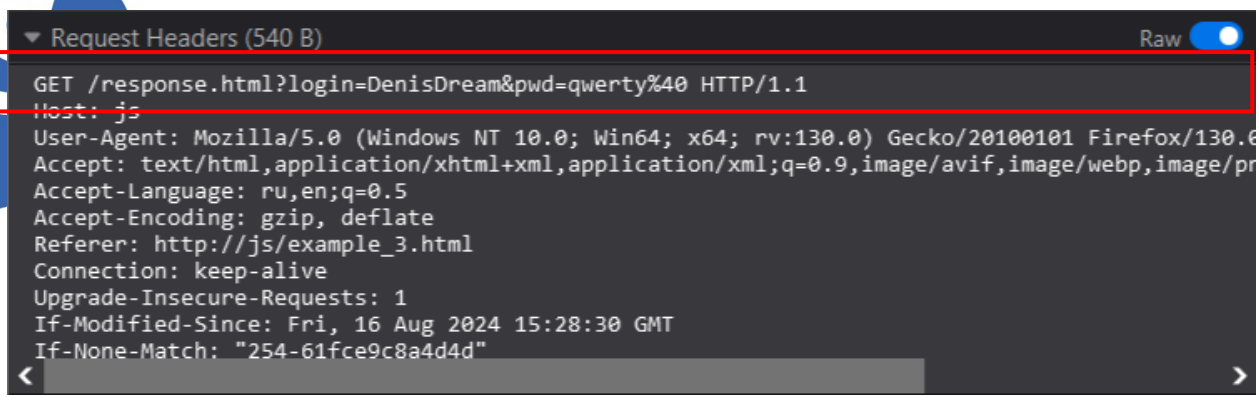
```
GET /async/lesson/example__.html HTTP/1.1
```

Остальные строки – **HTTP-заголовки** (**HTTP Headers**).

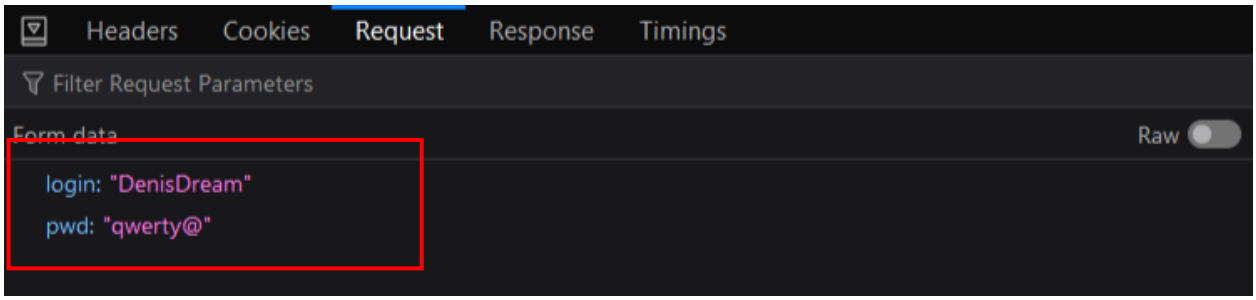
За HTTP-заголовками может находиться тело запроса. В теле запроса передаются **данные** клиента. Заголовки от данных отделяются **пустой строкой**.

> ***Важно***. **Пустая строка** – разделитель между **HTTP-заголовками** и **данными** клиента.

В зависимости от используемого метода **отправляемые на сервер данные** можно просмотреть в строке запроса:

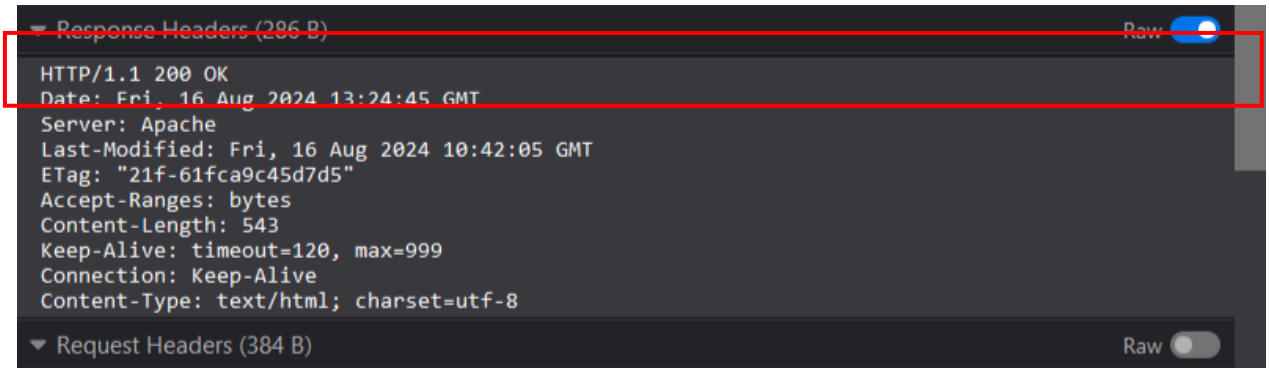


или на вкладке **Request**:



Получение HTTP-ответа

После отправки запроса браузер получает **HTTP-ответ**, который может выглядеть так:



Первая строка ответа — это **Status Line** (**Строка состояния**):

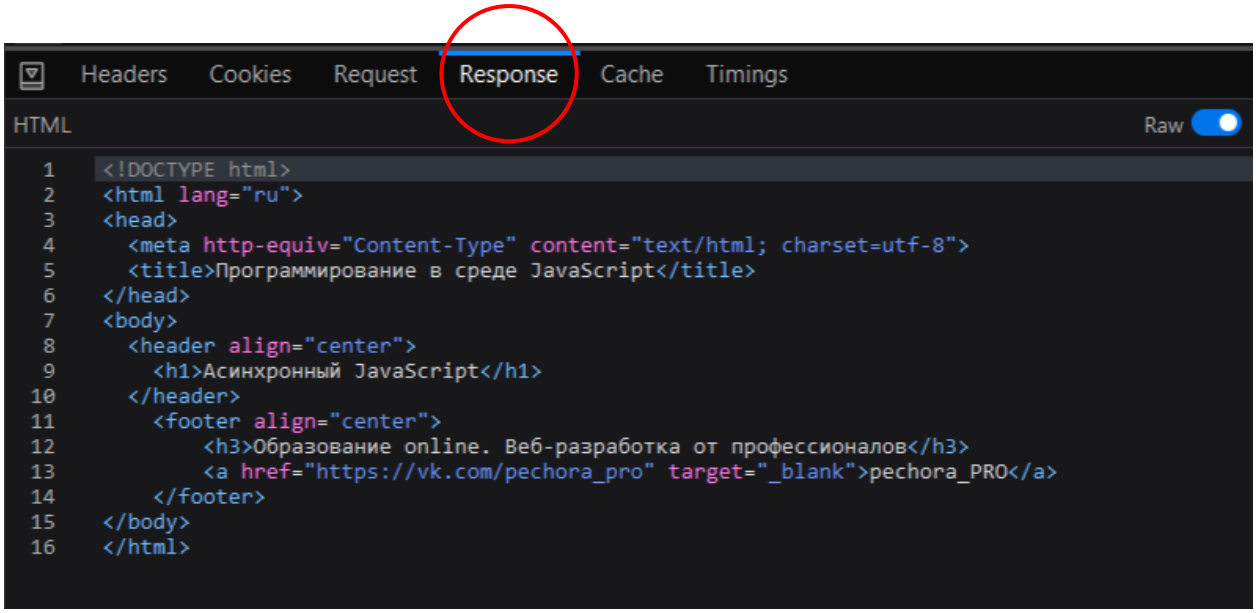
```
HTTP/1.1 200 OK
```

За строкой состояния следуют **HTTP-заголовки** (**HTTP Headers**).

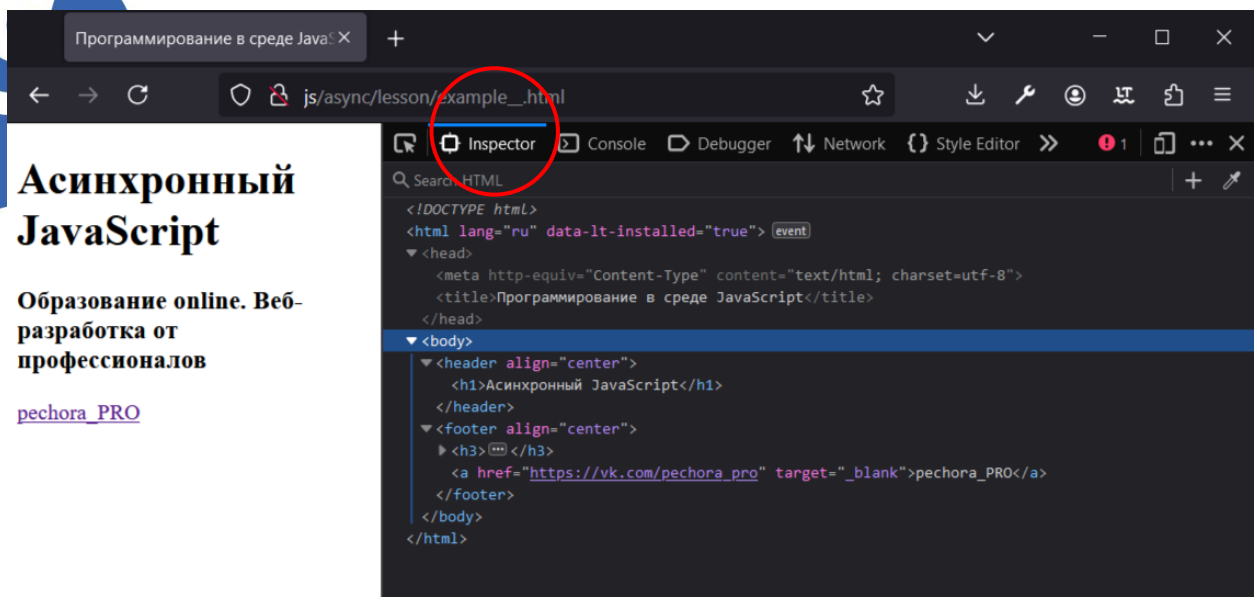
Конец HTTP-заголовков в ответе сервера – **пустая строка**. После этого может следовать тело документа – **содержимое** (контент) **документа**. Как правило контент документа – HTML-код для вывода в браузер.

> ***Важно***. **Пустая строка** – разделитель между **HTTP-заголовками** и **телом документа** отправляемым сервером.

Ответ сервера можно просмотреть на вкладке **Response**.



Когда пользователь просматривает **исходный код** веб-страницы в браузере (вкладка Inspector), он видит только часть ответа – **содержимое** документа, но **не заголовки** HTTP, хотя они фактически были переданы вместе.



Таким образом, в самом общем случае:

HTTP-запрос это:

- Request Line
- HTTP Headers
- Пустая строка
- Content

HTTP-ответ это:

- Status Line
- HTTP Headers
- Пустая строка
- Content

> ***Важно***. Структура запроса и ответа **зависит от используемого в запросе метода**. Разобраться со структурой запроса при использовании разных методов – задача текущего урока.

HTTP-запросы отправляются на все внешние ресурсы

HTTP-запросы отправляются не только к файлу, URL которого введен в адресную строку браузера. Запросы отправляются на все внешние ресурсы.

Примечание. Вводом URL-адреса в браузер пользователь инициирует отправку запроса к внешнему сетевому ресурсу. Остальные запросы инициируются самим браузером, **по мере появления в документе соответствующих внешних ссылок**.

HTTP-запросы отправляются и принимаются для изображений, CSS-файлов, файлов JavaScript и т. д.

В качестве примера – запросы к внешним ресурсам, ссылки на которые используются практически в каждом HTML-документе.

```
<!-- ссылка на внешний ресурс - скрипт -->
<script src="script.js"></script>
<!-- ссылка на внешний ресурс - файл стилей -->
<link rel="stylesheet" href="style.css">
<!-- ссылка на внешний ресурс - изображение -->
<img src="image.jpg" width="200px">
```

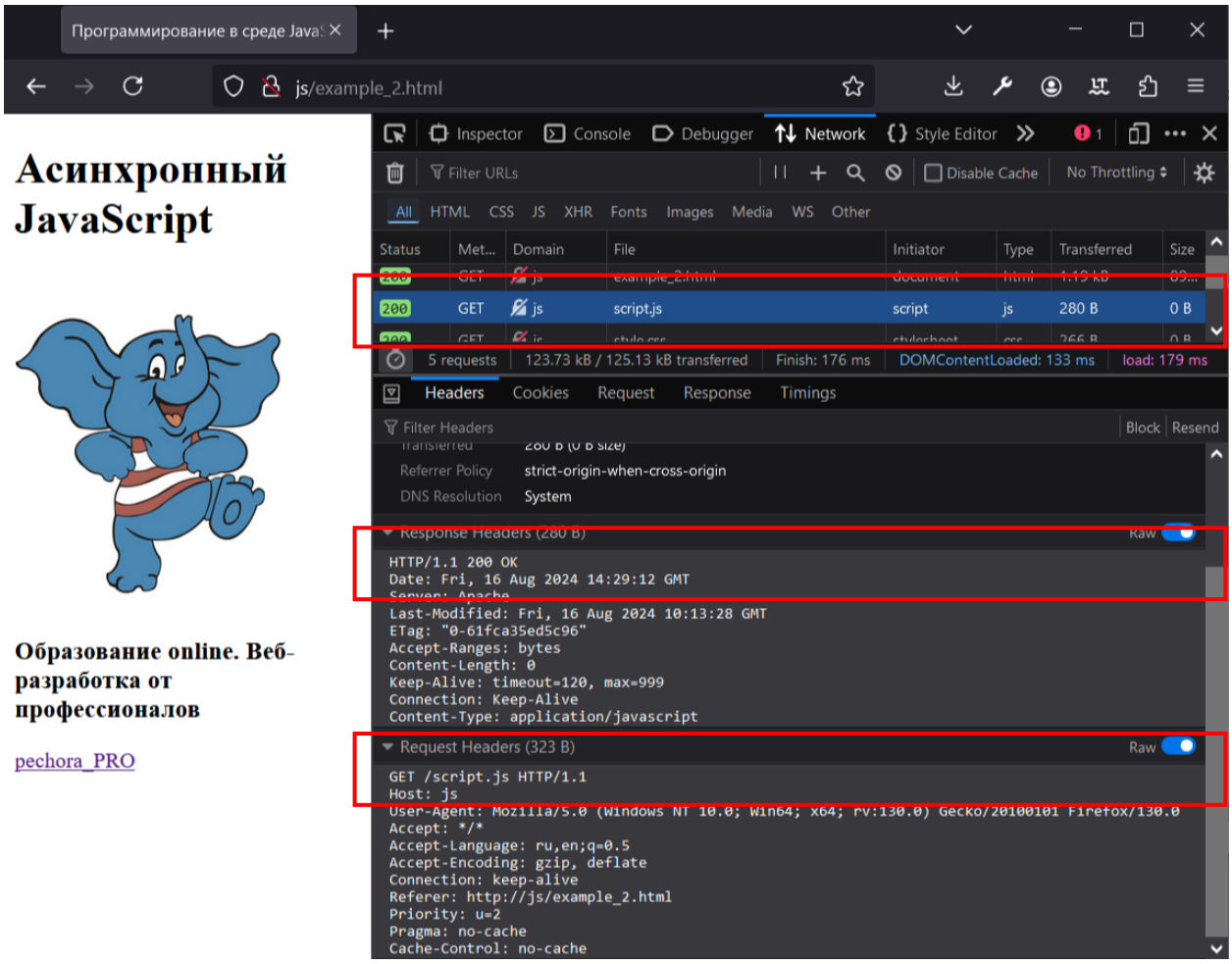
В демонстрационном примере **example_2** приведены стандартные примеры внешних ресурсов страницы.

example_2.

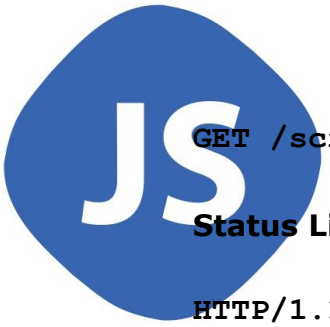
```
<!DOCTYPE html>
<html lang="ru">
    <head>
        <meta http-equiv="Content-Type" content="text/html;
        charset=utf-8">
        <title>Программирование в среде JavaScript</title>
        <!-- ссылка на внешний ресурс - скрипт -->
        <script src="script.js"></script>
        <!-- ссылка на внешний ресурс - файл стилей -->
```

```
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <header align="center">
        <h1>Асинхронный JavaScript</h1>
    </header>
    <!-- ссылка на внешний ресурс - изображение -->
    <img src="image.jpg" width="200px">
    <!-- ... -->
```

На следующем скриншоте я просмотрел обмен заголовками при обращении к внешнему ресурсу – скрипту **script.js**.



Request Line (**Строка запроса**) запроса:



```
GET /script.js HTTP/1.1
```

Status Line (**Строка состояния**) ответа:

```
HTTP/1.1 200 OK
```

Прежде чем пользователь увидит полностью загруженную страницу, браузер может отправить на сервер десятки HTTP-запросов.

> ***Примечание***. Для сервера HTTP-заголовки и HTML-код содержимого документа – **обычный текст**, который отправляется клиенту (браузеру). Браузер понимает, что означает каждый из заголовков и каким образом необходимо интерпретировать поступающий код для построения **DOM**.

Рассмотрим структуру заголовков более подробно.

Структура HTTP-запроса

Первая строка HTTP-запроса называется **Request Line** (**Строка запроса**) и состоит из трёх частей:

- **method** – указывает, какой это запрос. Наиболее распространённые методы **GET**, **POST** и **HEAD**.
- **path** – является частью **URL-адреса**, который идёт после **host** (домена).
- **protocol** – содержит название (HTTP) и версию протокола, в современных браузерах обычно 1.1.

Пример:

```
GET /async/lesson/example__.html HTTP/1.1
```

- method – **`GET`**
- path – **`/async/lesson/example__.html`**
- protocol – **`HTTP/1.1`**





Остальная часть запроса содержит HTTP-заголовки в виде пары **Name: Value** в каждой строке. Заголовки содержат различную информацию о HTTP-запросе и браузере пользователя. Например, строка **User-Agent** предоставляет информацию о версии браузера и операционной системе, которую использует пользователь (подробнее о заголовках дальше).

Данные Cookie также передаются внутри HTTP-заголовка.

Большинство заголовков являются **необязательными**, HTTP-запрос мог быть, например, таким:

```
GET /async/lesson/example__.html HTTP/1.1
Host: js
```

Даже такой запрос приведет к правильному ответу веб-сервера.

Методы запроса

Метод — это действие, которое клиент ждёт от сервера. Например, отправить ему HTML-страницу сайта или скачать документ. Протокол HTTP не ограничивает количество разных методов, но среди программистов есть негласная договоренность использовать только три основных, наиболее распространенных метода запроса: **GET**, **POST** и **HEAD**.

- **GET** — получить данные с сервера.
- **POST** — отправить данные на сервер. Например, сообщение в Telegram или новый файл в VK.



- **HEAD** — получить только метаданные об HTML-странице сайта. Метаданные находятся в head-теге HTML-файла.

Подробный анализ методов выполним при реализации асинхронного обмена данными в следующих уроках текущей темы.

Метод GET

Это основной метод, используемый для извлечения HTML, изображений, JavaScript, CSS и т. д. С использованием этого метода запрошено большинство данных, загружаемых в браузер.

Веб-форма демонстрационного примера **example_3** настроена на передачу данных методом GET.

example_3.

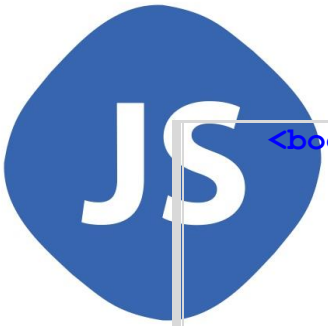
```
<form method="GET" action="response.html">
    <input type="text" name="login" value="DenisDream"><p>
    <input type="text" name="pwd" value="qwerty@"><p>
    <input type="submit">
</form>
```

Все формы урока в качестве **обработчика** используют файл **response.html**.

response.html

```
<!DOCTYPE html>
<html lang="ru">
    <head>
        <meta http-equiv="Content-Type" content="text/html;
        charset=utf-8">
        <title>Программирование в среде JavaScript</title>
    </head>
```

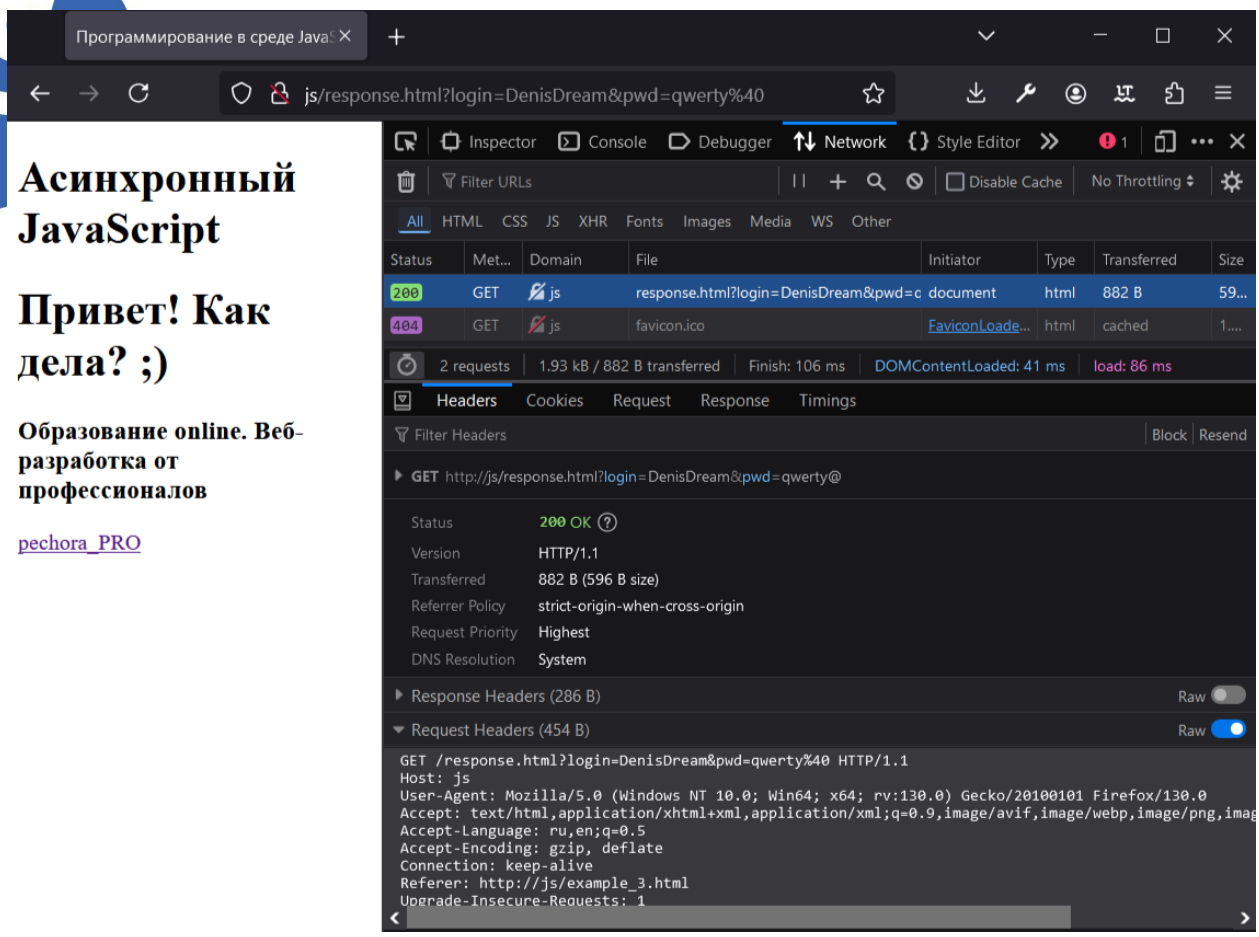


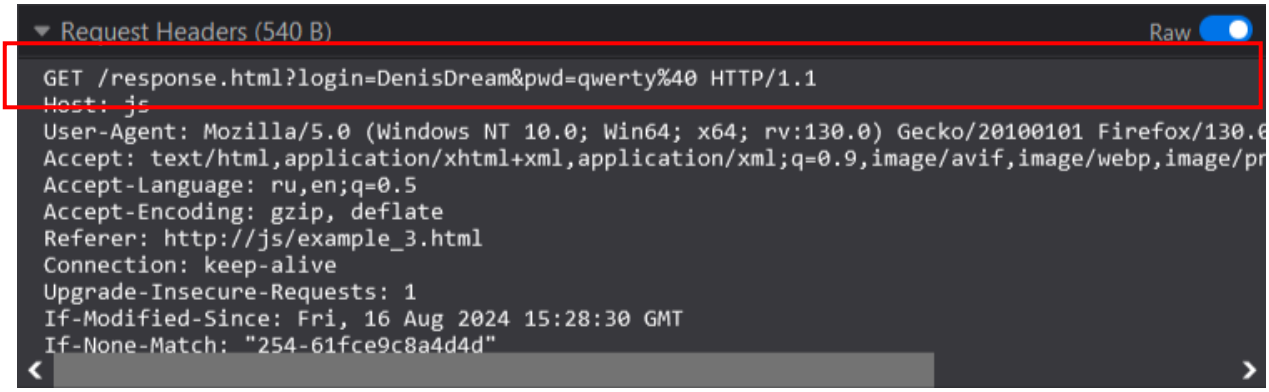


```
    <body>
        <header align="center">
            <h1>Асинхронный JavaScript</h1>
        </header>
        <h1>Привет! Как дела? ;)</h1>
        <footer align="center">
            <h3>Образование online. Веб-разработка от
            профессионалов</h3>
            <a href="https://vk.com/pechora_pro"
            target="_blank">pechora_PRO</a>
        </footer>
    </body>
</html>
```

Данные формы добавляются в строку запроса.





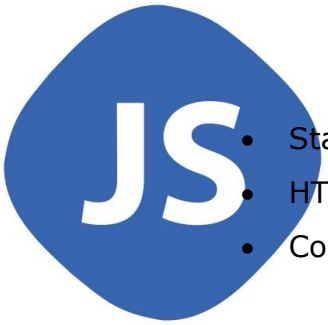


Ответом сервера на GET-запрос является, как правило, **запрашиваемый ресурс**.

Таким образом, **GET-запрос** состоит из:

- Request Line;
- HTTP Headers.

Ответ на **GET-запрос** состоит из:



- Status Line;
- HTTP Headers;
- Content.

Метод HEAD

HEAD идентичен GET, за исключением того, что сервер **не возвращает** содержимое (контент) HTTP-ответа. Когда вы отправляете запрос HEAD, это означает, что вас интересуют только **код ответа** и **HTTP-заголовки**, а не сам документ (контент документа не отправляется).

> ***Важно***. Когда вы отправляете запрос HEAD, это означает, что вас интересуют только **код ответа** и **HTTP заголовки**, а не сам документ.

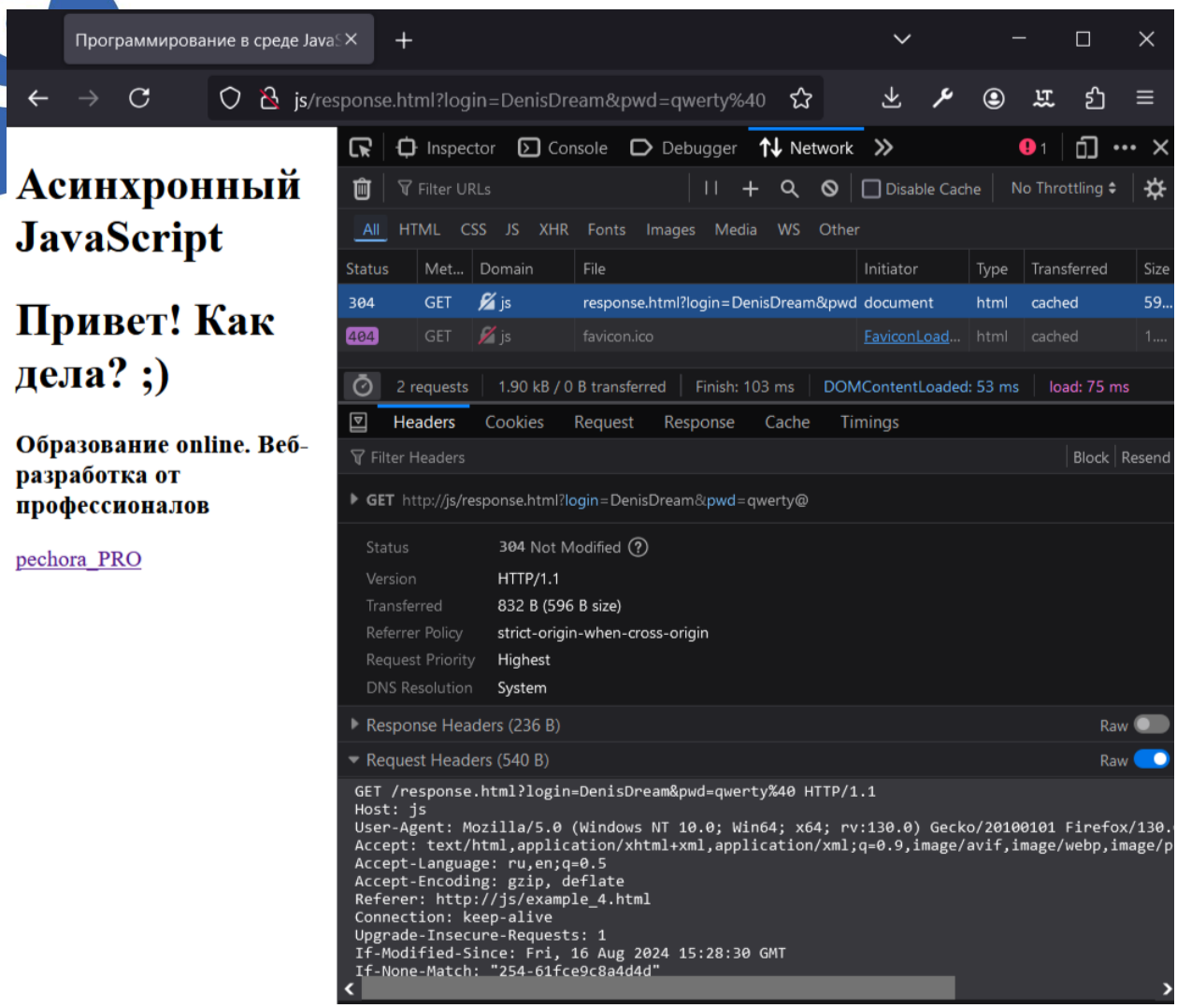
С помощью этого метода браузер может проверить, был ли документ изменён для целей кеширования. Он также может проверить, существует ли документ вообще.

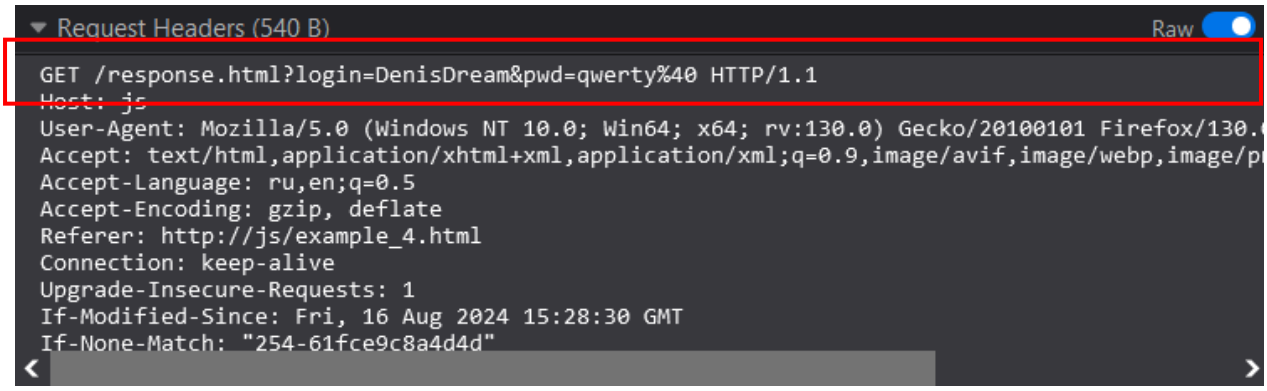
Форма демонстрационного примера **example_4** настроена на выполнение запроса методом HEAD.

example_4.

```
<form method="HEAD" action="response.html">
    <input type="text" name="login" value="DenisDream"><p>
    <input type="text" name="pwd" value="qwerty@"><p>
    <input type="submit">
</form>
```

Данные формы добавлены в строку запроса.

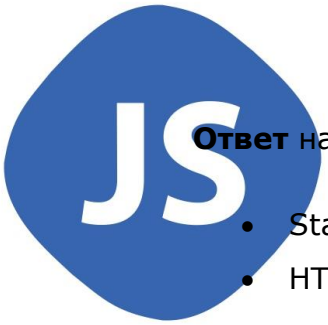




Содержимое ресурса при HEAD запросе **отправляться не будет**.

Таким образом, **HEAD-запрос** состоит из:

- Request Line;
- HTTP Headers.



Ответ на **HEAD-запрос** состоит из:

- Status Line;
- HTTP Headers.

Метод POST

Несмотря на возможность отправлять данные на сервер с помощью формы с методом GET или строки запроса, во многих случаях **POST** будет предпочтительнее. Отправка больших объёмов данных с помощью GET нецелесообразна и имеет ограничения. Кроме того, передача конфиденциальных данных в теле запроса предпочтительнее по соображениям безопасности. Веб-формы чаще всего отправляются методом POST.

В демонстрационном примере **example_5** изменим метод тестируемой формы на POST.

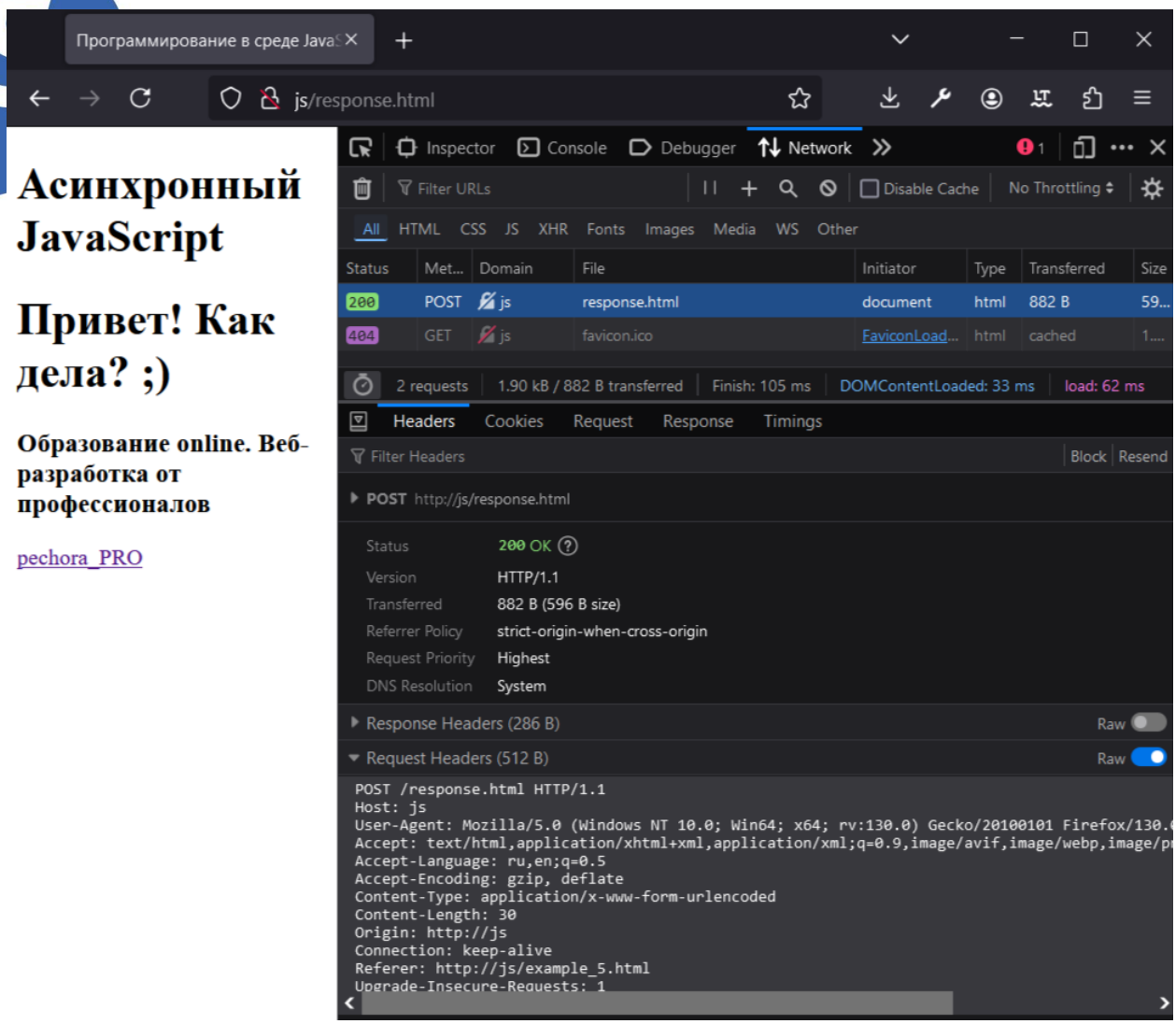
example_5.

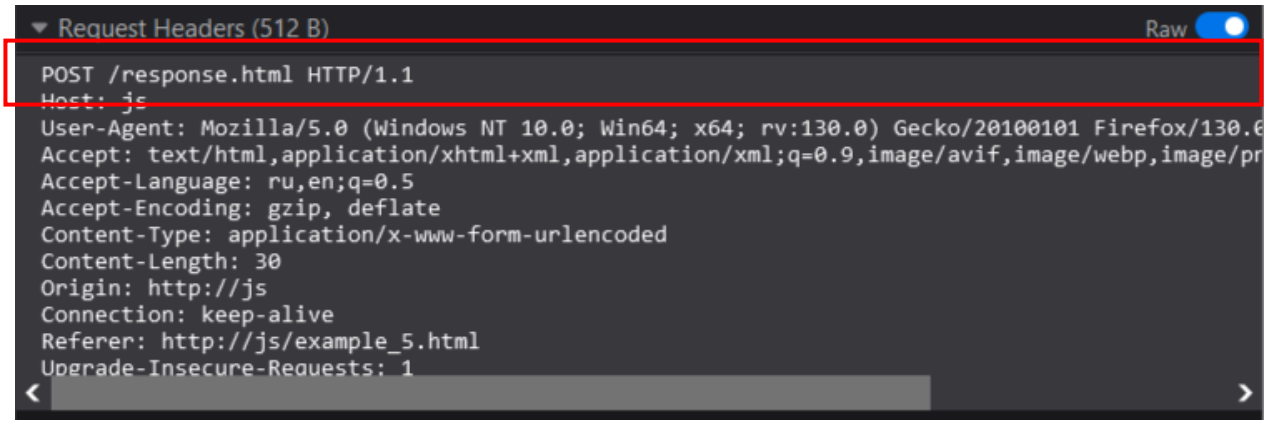
```
<form method="POST" action="response.html">
    <input type="text" name="login" value="DenisDream"><p>
    <input type="text" name="pwd" value="qwerty@"><p>
    <input type="submit">
</form>
```

Обратите внимание, в первой строке нет данных запроса.

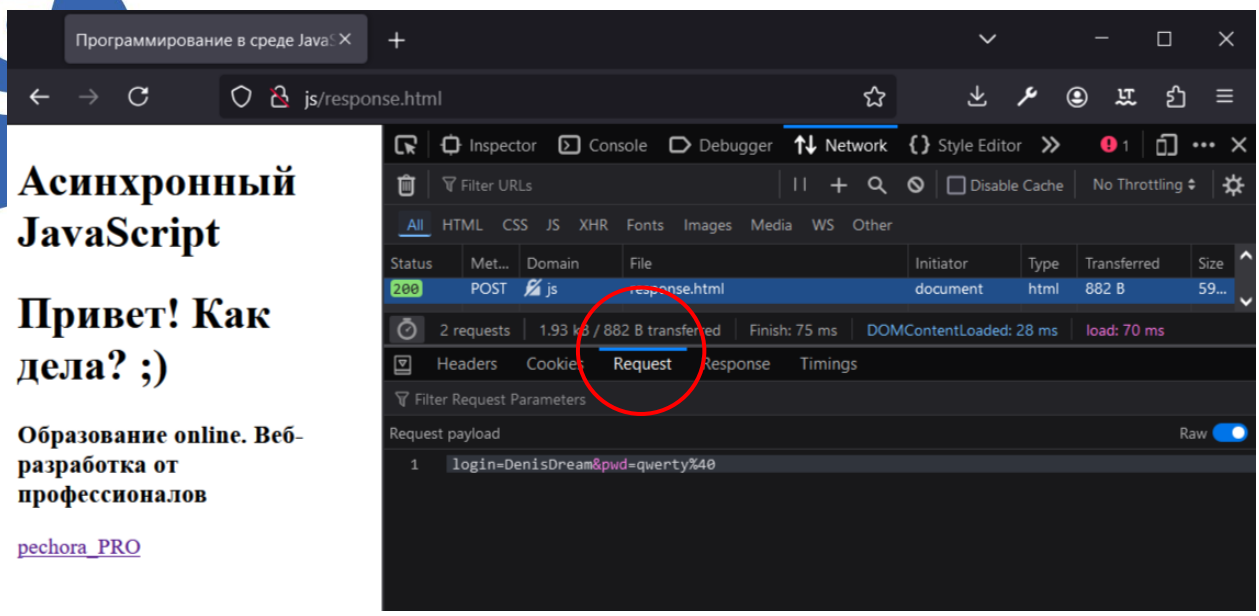
Зато в запросе появились заголовки **Content-Type** и **Content-Length**, которые предоставляют информацию об **отправляемых данных формы**.







Данные формы передаются после заголовков **в теле документа,** в том же формате, что и строка запроса. Просмотреть данные можно на вкладке **Request**.



Ответом сервера на POST-запрос является, как правило, **результат работы запрашиваемого скрипта**.

Таким образом, **POST-запрос** состоит из:

- Request Line;
- HTTP Headers;
- Content.

Ответ на **POST-запрос** состоит из:

- Status Line;
- HTTP Headers;
- Content.

Запросы POST метода также могут быть сделаны через AJAX, приложения, cURL и т. д.

Структура HTTP-ответа

После того, как браузер отправляет HTTP-запрос, сервер отвечает **HTTP-ответом**. Первая строка ответа называется **Status Line** (**Строка состояния**) и состоит:

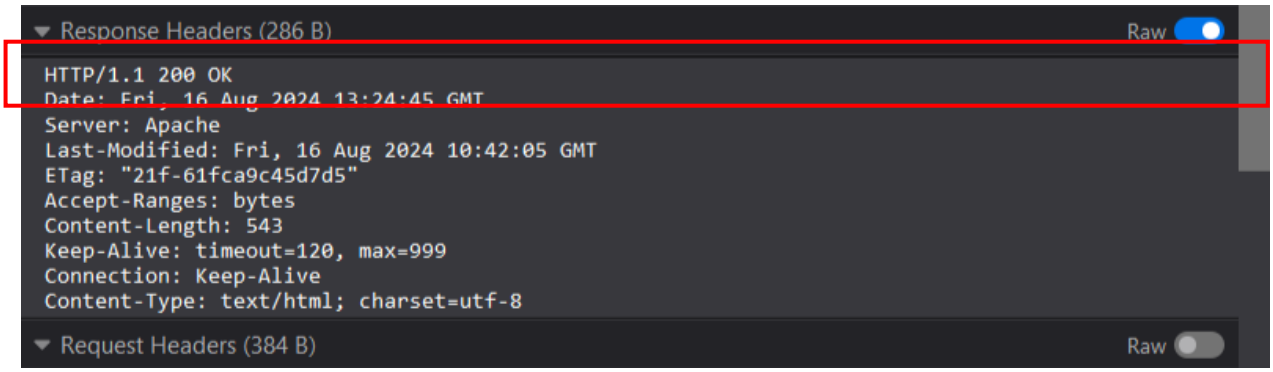
- **protocol** – обычно это снова HTTP/1.x или HTTP/1.1 на современных серверах.
- **status code** – это код состояния. Код 200 означает, что запрос был успешным и сервер вернёт содержимое запрошенного документа сразу после заголовков ответа.
- **status message** — краткое описание кода состояния.

Пример:

```
HTTP/1.1 200 OK
```

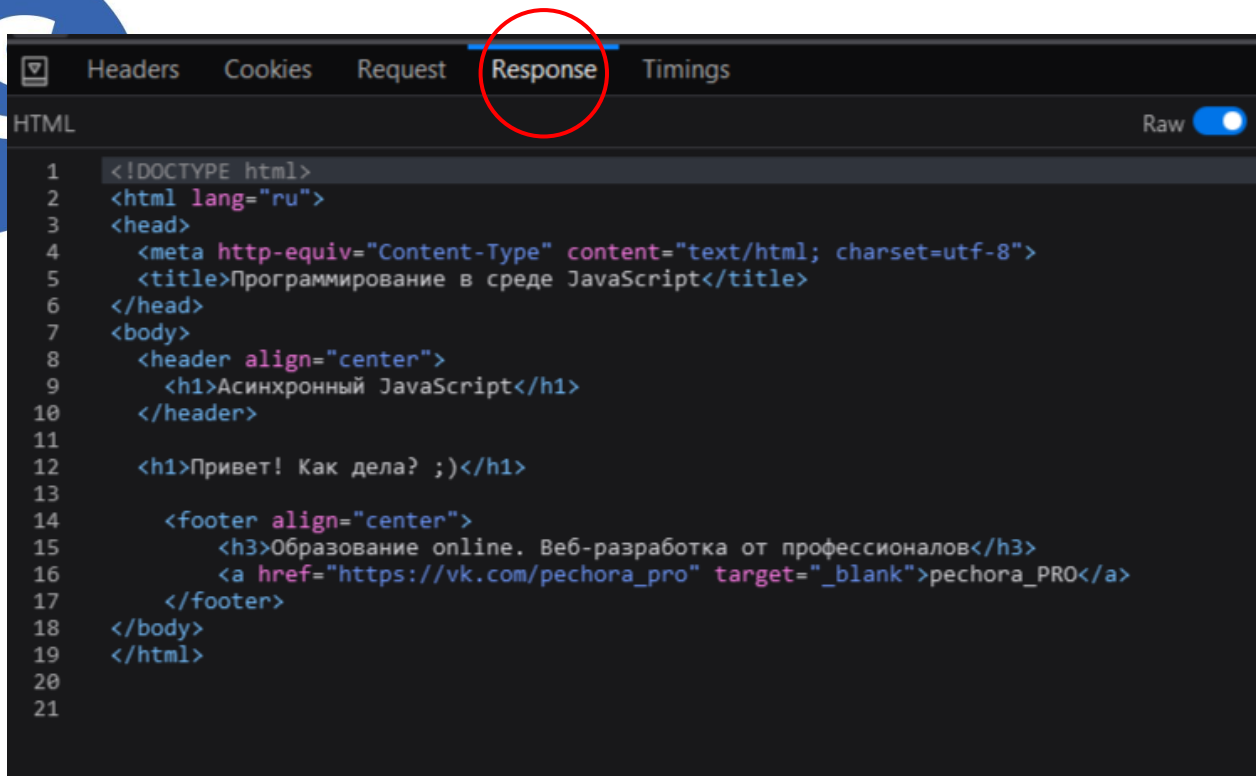
- protocol – **`HTTP/1.1`**
- status code – **`200`**
- status message – **`OK`**

Без учета контентной части заголовок ответа выглядит следующим образом:



Остальная часть ответа содержит заголовки так же, как HTTP-запрос. Значения заголовков могут содержать информацию о программном обеспечении сервера, дате последнего изменении страницы/файла, типе mime и прочее. Большинство заголовков являются необязательными.

В большинстве ответов после заголовков будет передаваться **тело документа**.



Самый простой ответ может выглядеть следующим образом:

```
HTTP/1.1 200 OK

Content-Type: text/html; charset=utf-8

Content-length: 85


<html>

<head>

<title>HTTP Headers</title>

</head>

<body>

Привет! Как дела?

</body>

</html>
```



Обратите внимание на обязательную **пустую строку** после HTTP-заголовков и параметр **Content-Length** – длину контента ответа (ну, если я не ошибся в подсчете ;)

Коды статуса HTTP

Статус ответа содержит версию HTTP-протокола, который клиент указал в HTTP-запросе. После этого код статуса ответа и словесное описание статуса ответа.

Всего статусов в спецификации HTTP 1.1 — 40. Вот самые популярные из них:

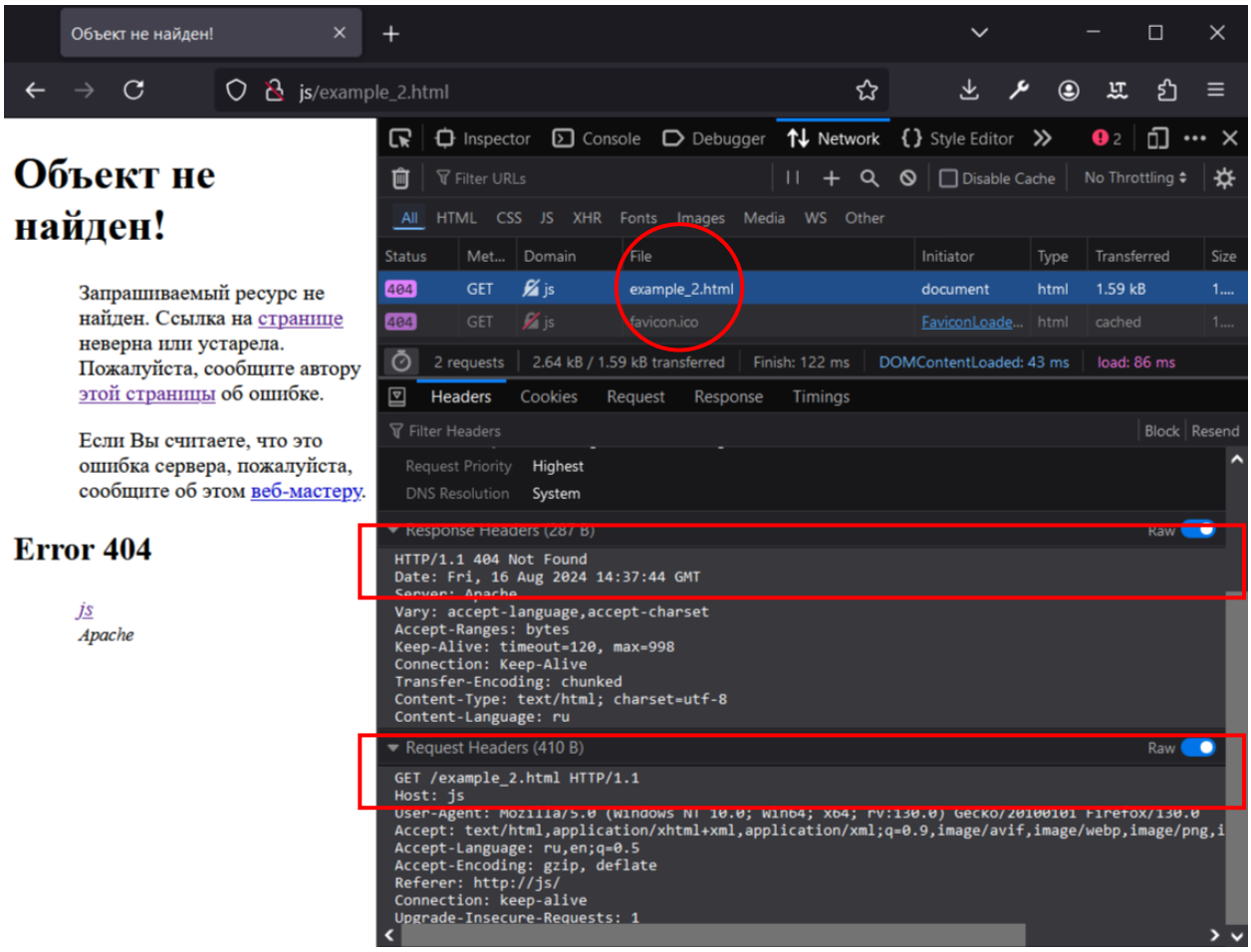
- **200 OK** — данные успешно получены;
- **201 Created** — запрос успешный, а данные созданы. Его используют, чтобы подтверждать успех запросов PUT или POST;
- **302 (or 307) Moved Temporarily & 301 Moved Permanently** – используются для **перенаправления** браузера. 302 и 301 обрабатываются браузером очень похоже, но они могут иметь различные значения для поисковых систем. Например, если ваш сайт не готов для обслуживания, вы можете перенаправить его в другое место с помощью 302. Поисковая система продолжит проверку вашей страницы в будущем. Но если вы перенаправите с использованием 301, это сообщит поисковым системам, что ваш сайт переехал в это место навсегда;
- **400 Bad Request** — означает неверно сформированный запрос. Обычно это случается в связке с запросами POST и PUT, когда данные не прошли проверку или представлены в неправильном формате;
- **401 Unauthorized** — ресурс требует аутентификации;
- **403 Forbidden** – код может быть отправлен, если пользователю не разрешен доступ к странице. Часто происходит, при попытке открыть URL-адрес для папки, в которой нет индексной страницы. Если параметры сервера не позволяют отображать содержимое папки, генерируется ошибка 403;
- **404 Not Found** — не удалось найти запрашиваемый ресурс;
- **405 Forbidden** — указанный метод HTTP не поддерживается для запрашиваемого ресурса;

- **409 Conflict** — произошёл конфликт запроса. Например, когда клиент хочет создать дважды данные с помощью запроса PUT;
- **500 Internal Server Error** — означает ошибку со стороны сервера. Возможные причины возникновения ошибки:
  - ошибки в синтаксисе файла .htaccess;
  - некорректное выполнение скриптов на сервере;
  - проблемы с правами доступа к файлам и папкам.

Выполним пару экспериментов со статусом **404 Not Found**.

Вернемся к демонстрационному примеру **example_2**. Удалите HTML-файл **example_2.html** и обновите страницу.

Проанализируйте заголовки запроса-ответа **HTML-файла**.



Request Line (**Строка запроса**) запроса:

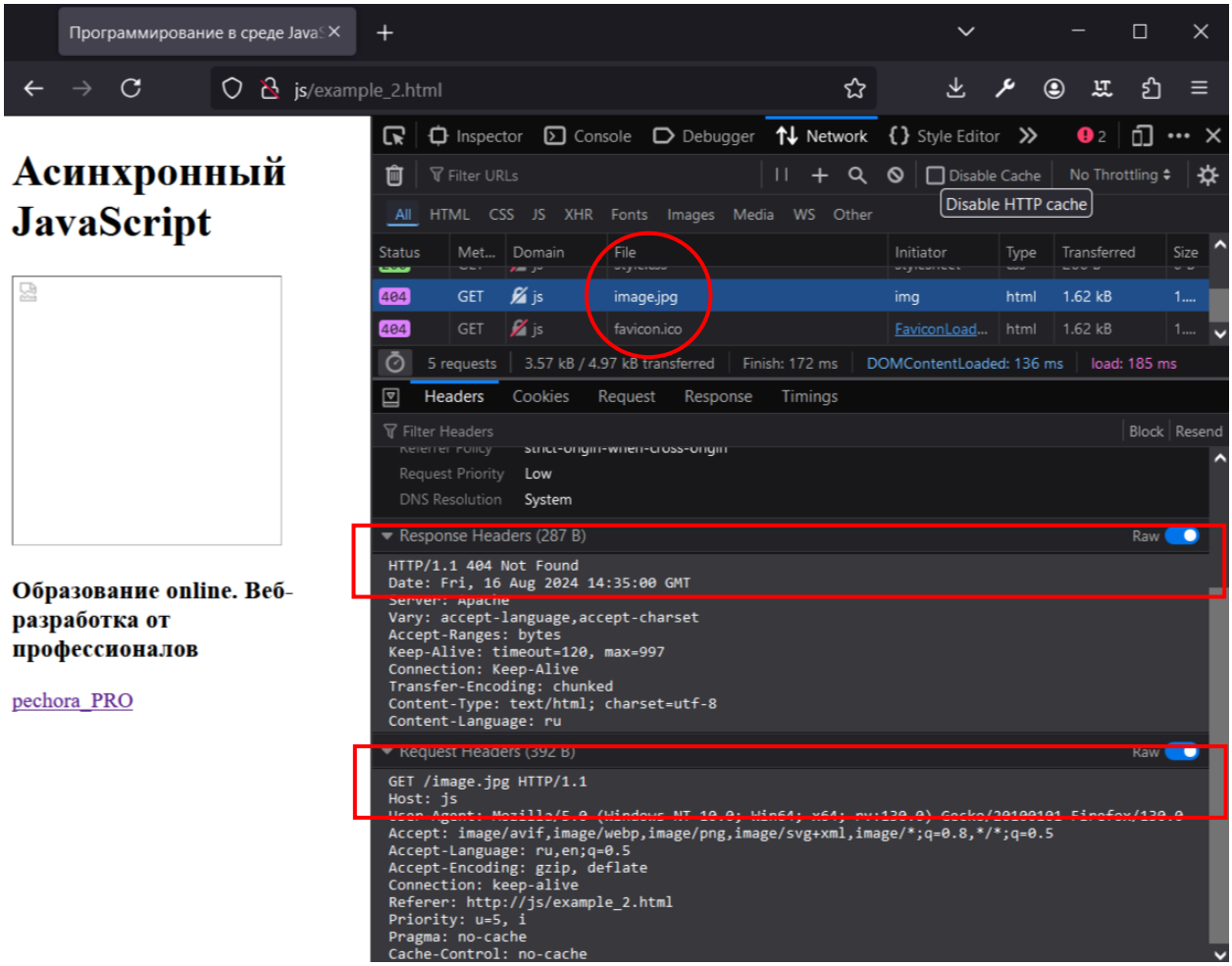
```
GET /example_2.html HTTP/1.1
```

Status Line (**Строка состояния**) ответа:

```
HTTP/1.1 404 Not Found
```

Восстановите файл. Теперь удалите файл изображения **image.jpg** и снова обновите страницу.

Проанализируйте заголовки запроса-ответа **изображения**.



Request Line (**Строка запроса**) запроса:

```
GET /image.jpg HTTP/1.1
```

Status Line (**Строка состояния**) ответа:

```
HTTP/1.1 404 Not Found
```

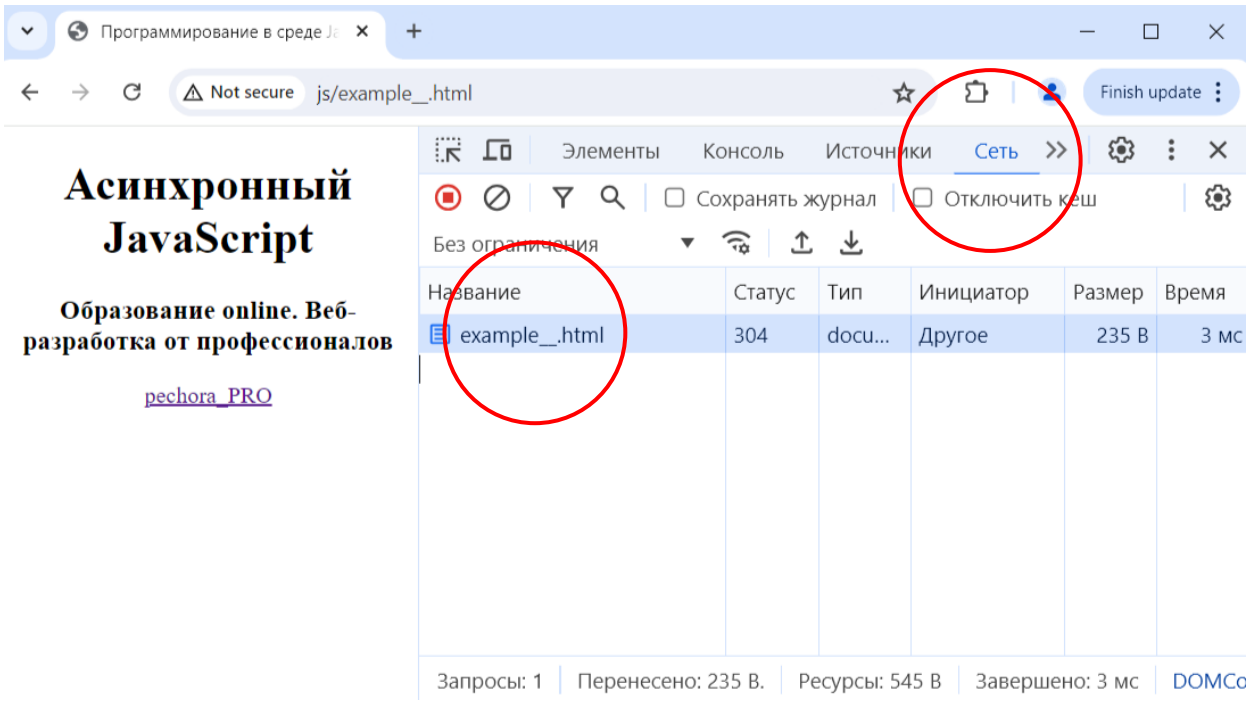
Заголовки в запросах HTTP

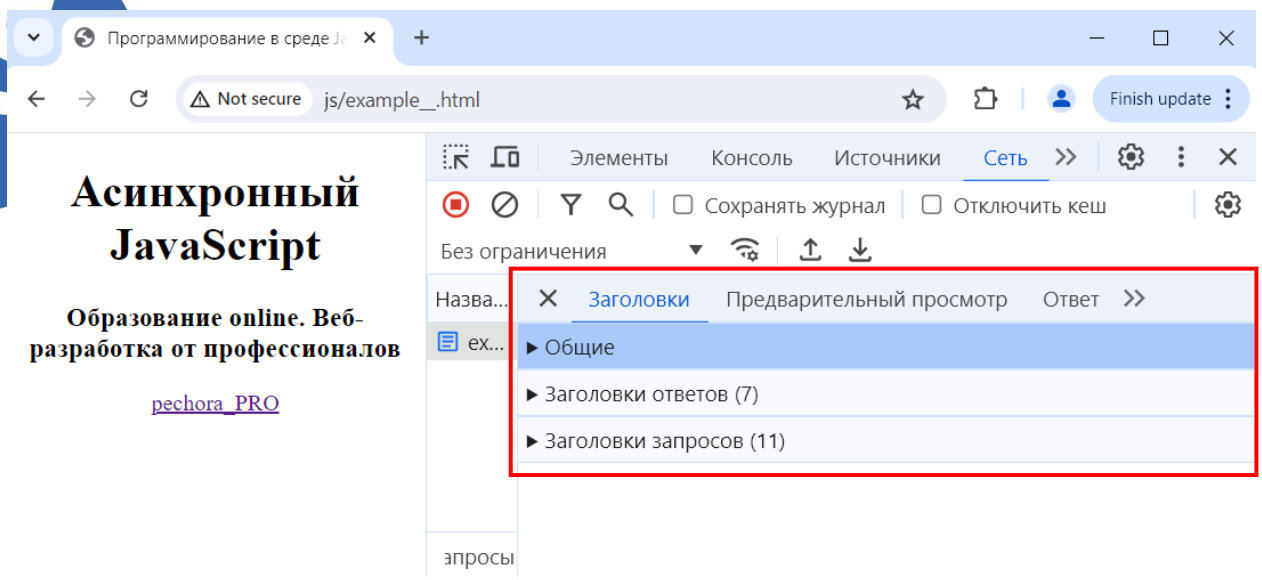
Рассмотрим **некоторые** из заголовков.

Заголовки могут быть сгруппированы по следующим контекстам:

- **General Headers** (**Общие заголовки**) применяется как к запросам, так и к ответам, но не имеет отношения к данным, передаваемым в теле. Должны включаться в любое сообщение клиента и сервера.
- **Request Headers** (**Заголовки запросов**) содержит больше информации о ресурсе, который нужно получить, или о клиенте, запрашивающем ресурс. Используются только в запросах клиента.
- **Response Headers** (**Заголовки ответов**) содержат дополнительную информацию об ответе, например его местонахождение, или о сервере, предоставившем его. Только для ответов от сервера.
- **Entity Headers** (**Заголовки сущности**) содержат информацию о теле ресурса, например его длину содержимого или тип MIME.

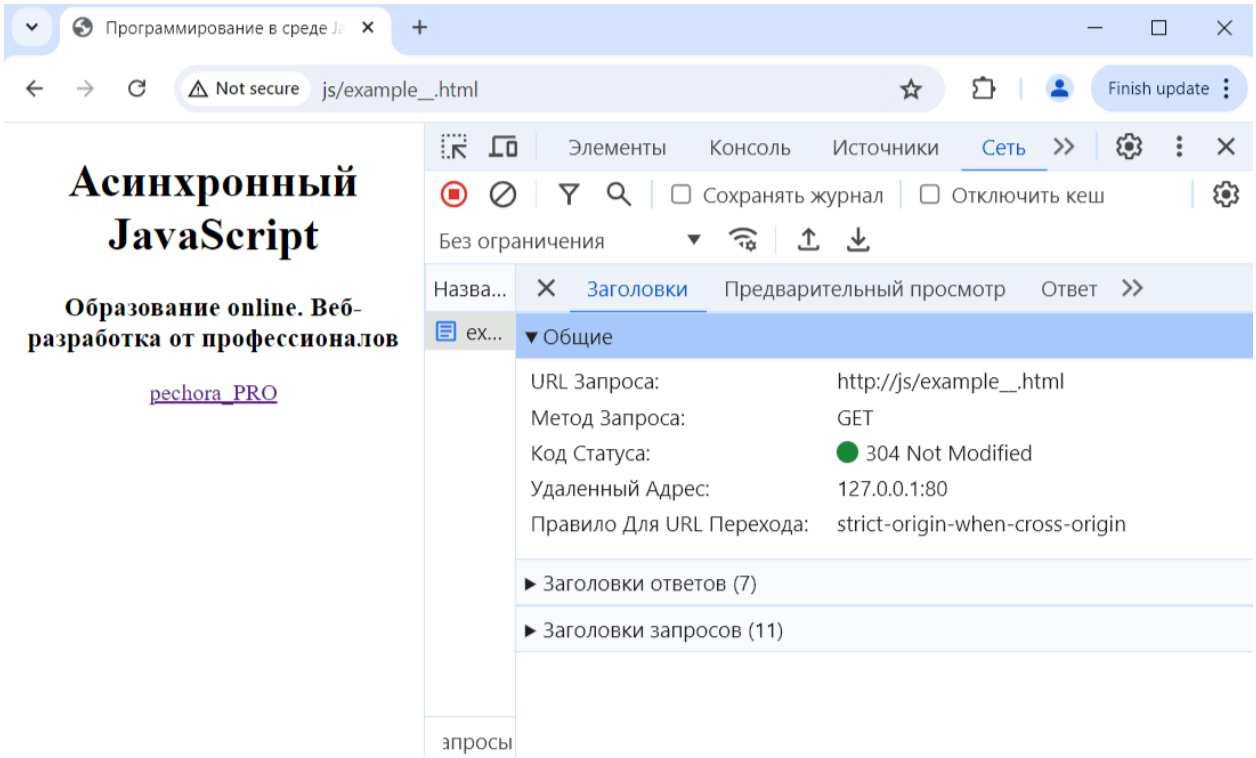
Заголовки рассмотрим на примере браузера **Google Chrome**.



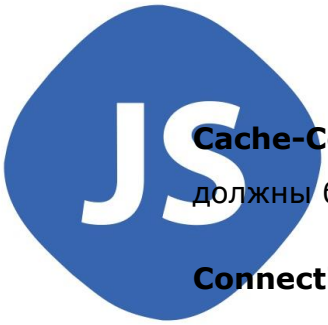


Общие заголовки (General Headers)

Общий заголовок (основной заголовок) — это заголовок, который используется и в запросе, и в ответе (в отличие от заголовка запроса и заголовка ответа), однако общий заголовок не описывает тело сообщения, как заголовок сущности.



Некоторые общие заголовки:



Cache-Control. Данное поле заголовка определяет директивы, которые должны быть выполнены системой кэширования.

Connection. Данное поле заголовка позволяет отправителю определить желаемые свойства для данного конкретного соединения и не должно связываться с прокси через текущее соединение.

Date. При работе с HTTP, время и дата должны быть представлены во времени по Гринвичу (Greenwich Mean Time – GMT) во всех случаях без исключений.

Transfer-Encoding. Данное поле указывает, какой тип преобразования был применён к телу сообщения. Используется для безопасного обмена данными между клиентом и сервером.

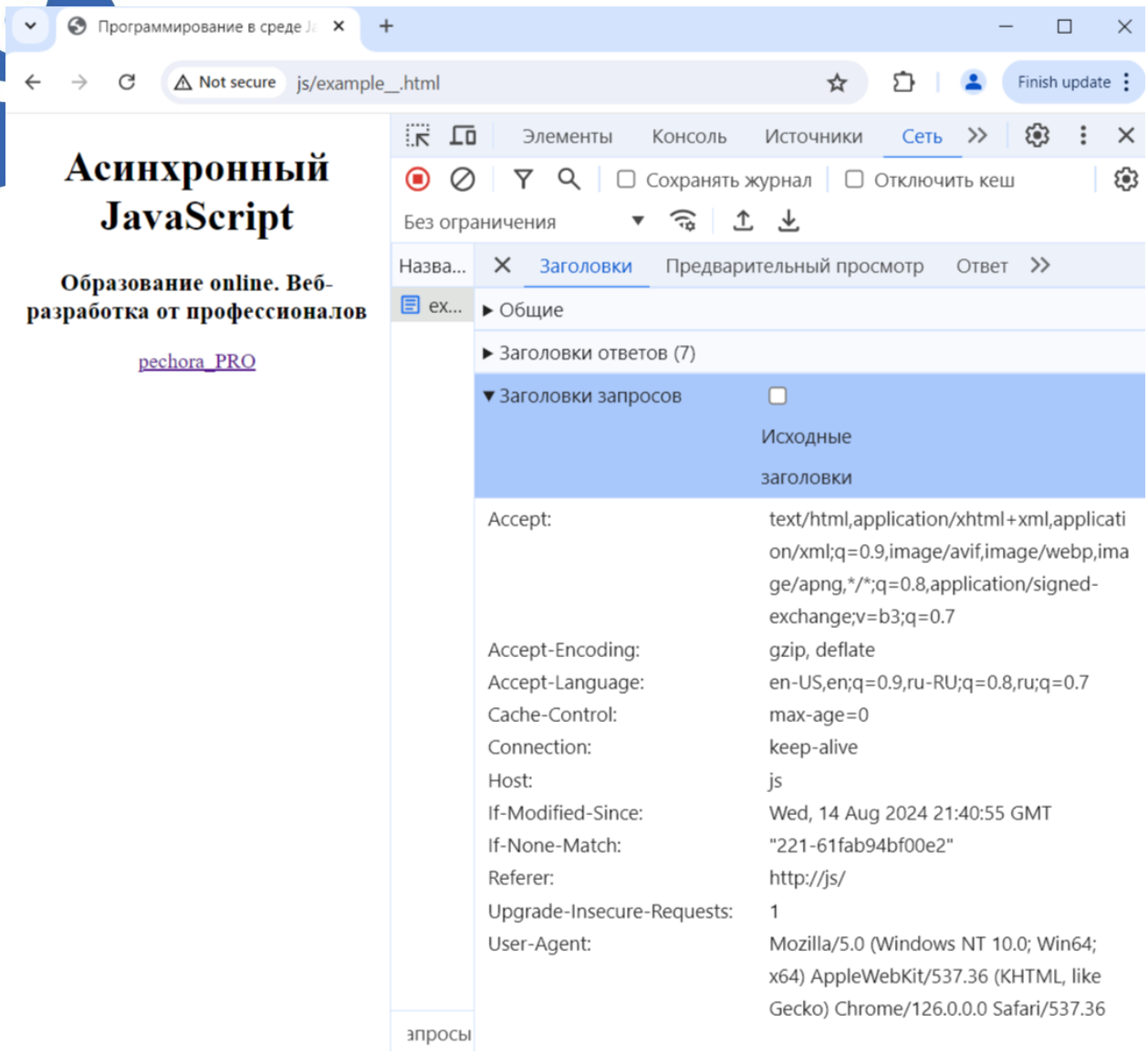
Upgrade. Данное поле позволяет клиенту определять дополнительные протоколы обмена данными, которые он поддерживает и хотел бы использовать в случае, если он поддерживается сервером.

Via. Это поле используется соединениями и прокси для указывания промежуточных протоколов и адресатов. Если данные идут не напрямую, а через прокси.

Warning. Данное поле используется для хранения дополнительной информации о статусе или преобразовании данных, которые могут не отображаться в самом сообщении.

Заголовки запросов (Request Headers)

Заголовок запроса содержит дополнительную информацию о ресурсе, который нужно получить, или о клиенте, запрашивающем ресурс.



Некоторые заголовки запросов:

Accept. Используется для определения определённых типов медиа файлов, которые применимы для ответа.

Accept-Charset. Указывает, какой набор символов приемлем для ответа.

Accept-Encoding. Указывает приемлемую для ответа кодировку контента.

Accept-Language. Указывает набор приемлемых языков для ответа.

Authorization. Содержит данные аутентификации пользователя для запрашиваемого ресурса. Содержит имя_пользователя:пароль, закодированные базовой 64-битной кодировкой.



Cookie. Содержит пару "имя-значение" информации, хранящейся по данному URL.

Expect. Указывает, что данный набор поведений сервера требуется клиентом.

From. Данное поле содержит email пользователя, который контролирует запросы пользователя.

Host. Определяет хост и порт запрашиваемого ресурса.

If-Match. Данное поле используется для того, чтобы сделать метод условным. Если мы хотим, чтобы метод выполнялся, только при соблюдении определённого условия.

If-Modified-Since. Используется для создания условий. Если запрашиваемая URL не была изменена с указанного времени, сущность будет возвращена сервером вместо ответа 304 (Not modified) без сообщения в теле.

If-None_Match. Используется для того, чтобы сделать метод условным. Выполняется только в том случае, если одно из переданных значений совпадает с тэгом, представленным ETag.

If-Range. Данное поле может быть использовано с условным GET для запроса определённой части данных.

Max-Forwards. Этот заголовок обеспечивает механизм с методами TRACE и OPTIONS для ограничения количества прокси или соединений, которые могут передавать запрос следующему серверу.

Proxy-Authorization. Данный заголовок позволяет клиенту определять себя (или своего пользователя) для прокси, который требует аутентификации.

Range Поле заголовка Range определяет диапазон запрашиваемого из документа контента в байтах.

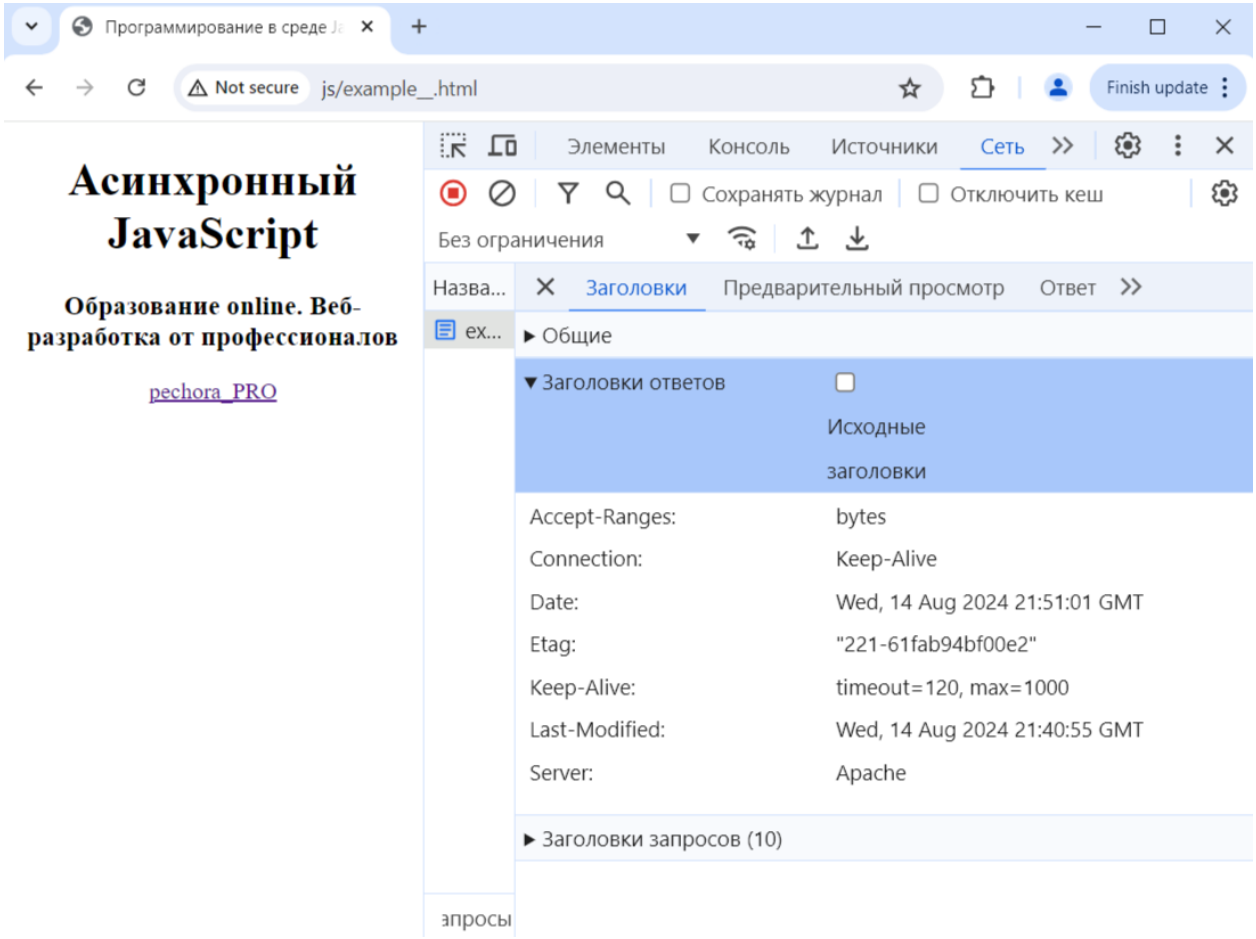
Referer. Данное поле заголовка позволяет клиенту определять URI ресурса, с которого пришёл запрос на URL.

User-Agent. Данное поле заголовка содержит информацию об агенте пользователя, посылающего запрос.



Заголовки ответов (Response Headers)

Заголовок ответа содержит дополнительную информацию об ответе, например о местонахождении ресурса, или о сервере, предоставившем ресурс.



Некоторые заголовки ответов:

Accept-Ranges. Позволяет серверу указывать принятие им диапазона запросов.

Age. Передаёт ожидания относительно времени (в секундах), прошедшего с момента ответа, который был сгенерирован сервером.

Etag. Поле заголовка ETag обеспечивает текущее значение тэга сущности для варианта, на который был сделан запрос.

Location. Используется для перенаправления клиента на URL, отличную от запрашиваемой.



Proxy-Authenticate. Данное поле заголовка должно быть включено как часть ответа 407 (Proxy Authentification Required).

Retry-After. Может быть использовано вместе с ответом 503 (Service Unavailable), чтобы указать, как долго сервис будет недоступен для запросов.

Server. Содержит информацию о ПО, которое используется сервером, обрабатывающем запрос.

Set-Cookie. Содержит пары значений "имя/значение", информации по текущему URL. Возможные значения, которые можно настроить:

- **Comment**. Для определения любого комментария, связанного с cookie.
- **Domain**. Определяет домен, для которого cookie валидны.
- **Expires**. Дата, после которого cookie не действительны. Если пусто, то истекают сразу после закрытия браузера клиентом
- **Path**. Определяет подмножество URL, к которым применимы cookie.
- **Secure**. Даёт указанию агенту пользователя возвращать cookie только под защищённым соединением

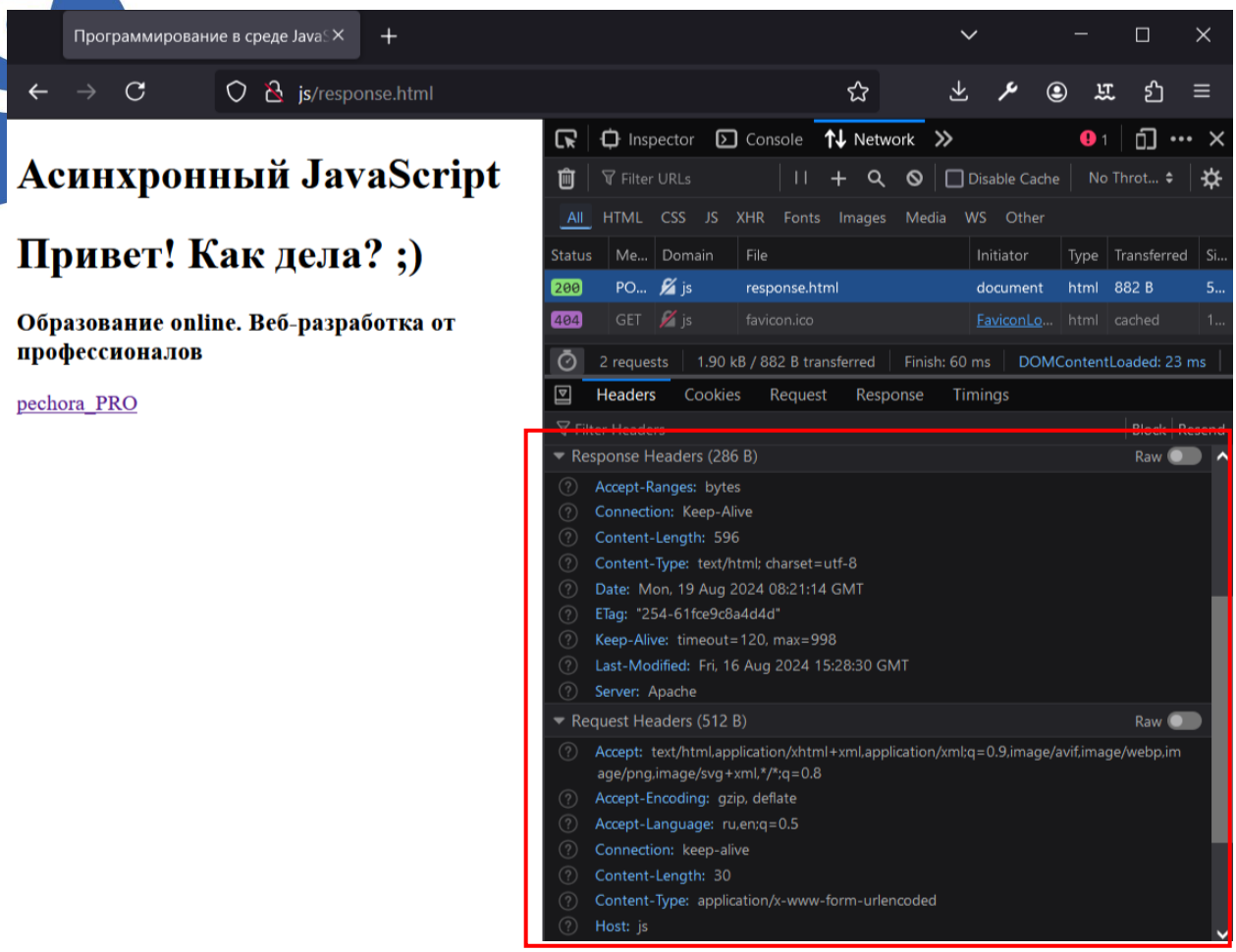
WWW-Authenticate. Должен быть включён в ответ 401 (Unauthorized).

Заголовки сущности (Entity Headers)

Заголовок сущности — это HTTP Header, описывающий **содержимое тела сообщения**. Заголовки сущности используются в HTTP-запросах и ответах. Заголовки, такие как **Content-Length**, **Content-Language**, **Content-Encoding** являются заголовками сущности.

Даже если заголовки сущности не являются заголовками запросов или ответов, они часто **включаются в эти понятия**.

На следующем скриншоте (example_4) заголовки **Content-Length** и **Content-Type** будучи заголовками **сущности**, включены и в заголовки **запроса,** и в заголовки **ответа**.



Некоторые заголовки сущности:

Allow. Содержит список методов, которые поддерживаются ресурсом, определяемом URL.

Content-Encoding. Используется как модификатор для типа медиа файлов. Если кодировка контента, запрашиваемая клиентом, не поддерживается сервером, то будет получен ответ 415 (Unsupported Media Type).

Content-Language. Определяет языки для определённой сущности. Если мы хотим использовать несколько языков, то мы должны указать их символы через запятую.

Content-Length. Данное поле заголовка указывает размер тела сущности десятичным числом (символы).



Content-Location. Используется для поддержки локации ресурса для сущности, помещённой в сообщение, когда сущность запрашивается из локации, отличной от URI запрашиваемого ресурса.

Content-MD5. Данный заголовок используется для поддержки шифрования MD5 для сущности.

Content-Type. Данное поле заголовка указывает тип медиа файла тело сообщения, переданного клиенту.

Expires. Указывает дату и время, после которого ответ считается устаревшим.

Last-Modified. Данное поле заголовка указывает дату и время, когда (по мнению сервера) был крайний раз изменён файл.

Дополнительный практикум в **самостоятельных работах**:

- Поиск отсутствующих внешних ресурсов используя режим разработчика браузера.
- Определение значения заголовков Content-Length предложенного примера.

P.S.

Для отработки и закрепления учебного курса **в расширенной платной версии** предоставляется следующий **раздаточный** материал.

К каждому **уроку** курса:

- Файлы **демонстрационного кода** (example).
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).

К каждой **теме** курса:

- **Практические работы**.



К **курсу**:

- **Дополнительный практикум.**

